

DTRAIN

A Project Report Submitted

In partial fulfilment of the requirements for the award of the degree of

Bachelor of Technology

In

Computer Science and Engineering

Submitted By:

Debjit Mitra (17600122046)

Soham De (17600122062)

Akash Poddar (17600122071)

Sourodip Ghosh (17600122040)

Under the guidance of

Ms. Agnimita Banerjee, Assistant Professor, Dept. of CSE



Department of Computer Science and Engineering

Hooghly Engineering & Technology College, Hooghly

Affiliated to

Maulana Abul Kalam Azad University of Technology,

West Bengal



2025-26

DTRAIN

A Project Report Submitted

*In partial fulfilment of the requirements for the award of the degree of
Bachelor of Technology*

In

Computer Science and Engineering

Submitted By:

Debjit Mitra (17600122046)

Soham De (17600122062)

Akash Poddar (17600122071)

Sourodip Ghosh (17600122040)

Under the guidance of

Ms. Agnimita Banerjee, Assistant Professor, Dept. of CSE



*Department of Computer Science and Engineering
Hooghly Engineering & Technology College, Hooghly*

*Affiliated to
Maulana Abul Kalam Azad University of Technology,
West Bengal*



2025-26



HOOGHLY ENGINEERING & TECHNOLOGY COLLEGE

Vivekananda Road, Pipulpati, P.O. & Dist-Hooghly (WB), Pin-712103

(Affiliated to Maulana Abul Kalam Azad University of Technology)

CERTIFICATE

This is to certify that the project entitled “**DTrain: Distributed Machine Learning Training Platform**” has been submitted to the **Department of Computer Science and Engineering, Hooghly Engineering & Technology College, Hooghly** in partial fulfilment for the award of Bachelor of Technology in Computer Science and Engineering by the following 4th Year 8th Semester students under the supervision of **Ms. Agnimita Banerjee, Assistant Professor, CSE Department**, during a period from **January, 2026 to June, 2026**.

Their performances were found to be satisfactory.

Student Name (University Roll No.)

1. Debjit Mitra (17600122046)
2. Soham De (17600122062)
3. Akash Poddar (17600122071)
4. Sourodip Ghosh (17600122040)

Ms. Agnimita Banerjee
Department of CSE

Dr. Sumanta Daw
HOD, Department of CSE

Prof. (Dr.) B. P. Pattanaik
Principal, HETC

ACKNOWLEDGEMENT

It gives us immense pleasure to announce the partial completion of our project on “**DTrain: Distributed Machine Learning Training Platform**” and we are pleased to acknowledge our indebtedness to all the persons who directly or indirectly contributed in the development of this work and who influenced our thinking, behaviour and acts during the course of study.

We are thankful to our respected departmental HOD Dr. Sumanta Daw and Coordinator Mr. Dibyendu Samanta who granted all the facilities of the college to us for the fulfilment of the project.

We are thankful and express our sincere gratitude to our project guide **Ms. Agnimita Banerjee** who gave his/her valuable time to us for the sake of our project. She helped us each and every aspect of our project both academically and mentally.

Finally, the team expressed their gratitude to our respected Principal sir Prof. (Dr.) Bhabani Prasanna Pattanaik without his support our project would not have seen the light of success.

Student Name (with Roll No.)

Signature

1. Debjit Mitra (17600122046)

.....

2. Soham De (17600122062)

.....

3. Akash Poddar (17600122071)

.....

4. Sourodip Ghosh (17600122040)

.....

4th Year, 8th Semester

Department of Computer Science and Engineering

Hooghly Engineering & Technology College

Academic Year: 2025-26

ABSTRACT

The escalating cost of cloud-based GPU compute has created a significant barrier for independent machine learning researchers and students in developing economies like India. A single training run of a BERT fine-tuning task on AWS EC2 (p3.2xlarge) costs approximately Rs.952, while a PyTorch CNN training job costs upwards of Rs.504 — prohibitive sums for individuals without institutional backing.

DTrain is a full-stack, peer-to-peer distributed AI model training platform that addresses this problem by connecting ML developers who need compute power with GPU owners who have idle machines, handling all logistics between them. Conceptually analogous to Airbnb for GPU compute, DTrain removes the need for any cloud account, infrastructure configuration, or per-hour billing uncertainty.

The platform is architected as four independent applications: a React/TypeScript user-facing web dashboard (frontend), an Express.js v5 + Socket.IO backend API server, a React-based worker dashboard (worker-ui), and an Electron desktop agent (electron-worker) that turns any gaming PC into a training node. The backend integrates MongoDB Atlas for persistent data storage, Redis for job queue pub-sub messaging, Supabase object storage for training ZIP files and model outputs, Stripe for payment escrow and worker payouts via Stripe Connect, and the Groq LLM API for AI-assisted job pricing.

The pricing engine is a two-layer system: a rule-based static analyser scans the submitted requirements.txt and training script, assigning complexity scores based on detected ML frameworks (PyTorch/TensorFlow: +40 pts, Hugging Face Transformers: +45 pts), dataset sizes, epoch counts, and neural network complexity patterns. A Groq LLM then provides an independent estimate, and the final tier price (selected from a Rs.10–Rs.500 ladder) is the maximum of both scores, preventing both under-pricing of heavy jobs and over-pricing of trivial ones.

The payment model uses Stripe's payment reservation (not immediate charge) approach: funds are reserved at job publication and only charged on successful completion, with automatic full refund on failure. Workers earn 80% of each job's tier price; the platform retains 20%. Worker payouts are handled through Stripe Connect.

Performance benchmarks derived from published GPU TFLOPS specifications show that DTrain provides a 4–6x speedup over a low-end consumer PC (GTX 1060-class hardware) for medium-to-heavy workloads, at 74–80% lower cost than equivalent AWS cloud instances — with zero setup time for users.

Key contributions: a working crowd-sourced compute marketplace with trust mechanisms; a novel two-layer AI pricing engine combining rule-based static analysis with LLM inference; real-time

training log streaming via Socket.IO from inside Docker containers; automated Docker container lifecycle management from within an Electron desktop application; and a complete payment escrow system with multi-party settlement.

INDEX

SL NO.	DESCRIPTION	PAGE
I.	ABSTRACT	4
II.	LIST OF FIGURES	7
1.	INTRODUCTION	9
2.	REVIEW OF LITERATURE	12
3.	PROPOSED WORK	14
4.	PROJECT MODULES & FEATURES	18
5.	PLATFORM & TECHNOLOGY	24
6.	TECHNICAL DETAILS	29
7.	SYSTEM IMPLEMENTATION & UI	37
8.	RESULT & DISCUSSION	47
9.	APPLICATION	51
10.	FUTURE SCOPE	53
11.	CONCLUSION	55
12.	REFERENCES	57

LIST OF FIGURES

SL NO.	DESCRIPTION	PAGE
1.1	Use Case Diagram — DTrain System Actors and Interactions	10
3.1	DTrain System Architecture — Four Component Layers	14
3.2	Job Lifecycle State Machine — Status Transitions	15
3.3	AI Pricing Engine — Two-Layer Scoring Flow	16
3.4	Payment Flow — Reserve, Charge, and Payout Sequence	17
4.1	Docker Job Execution Flow — Electron Agent to Container	20
4.2	Real-Time Data Flow Diagram — Log Streaming and Status Updates	21
4.3	Sequence Diagram — Full Job Lifecycle from User to Worker	22
5.1	DTrain Network Topology — Component Communication Paths	27
6.1	Entity-Relationship Diagram — MongoDB Collections	29
7.1	DTrain Landing Page / Hero Section	37
7.2	Sign In / Sign Up — Authentication Page	38

7.3	User Dashboard — Job List with Status Badges	38
7.4	Job Submission Form — ZIP Upload with Drag & Drop	39
7.5	AI Pricing Preview — Draft Price Before Payment	40
7.6	Job Detail — Live Log Streaming View During Training	40
7.7	Job Detail — Completed Job with Model Download Link	41
7.8	Wallet Page — Balance, Quick Top-Up, Transaction History	41
7.9	Stripe Embedded Checkout Modal — Wallet Top-Up	42
7.10	Active Workers Panel — Live Worker Status View	43
7.11	Worker Device Registration — Hardware Auto-Detection	43
7.12	Worker Dashboard — Earnings, Pending Jobs, Wallet Card	44
7.13	Worker Dashboard — Payout Request Form	45
7.14	Worker Running Jobs View — Active Job & Live Timer	45
7.15	Worker Job Completion — Earnings Credited	46
8.1	Training Time Comparison Across Platforms (hours)	47
8.2	Cost Per Job Comparison Across Platforms (INR)	48
8.3	DTrain Speed-Up Factor vs Low-End Consumer PC	49

INTRODUCTION

1.1 Background and Motivation

The field of machine learning (ML) and deep learning has witnessed unprecedented growth over the past decade. Modern ML models — from BERT-class transformers for natural language processing to convolutional neural networks for computer vision — demand substantial computational resources for training. A single fine-tuning run of a pre-trained language model such as BERT can require 3–4 hours on high-end hardware, translating to Rs.840–Rs.1120 on a reserved AWS EC2 p3.2xlarge instance at approximately Rs.280/hr. For a PyTorch CNN trained from scratch, the cost climbs further to over Rs.504 for a single job.

This compute cost barrier creates a stark inequity in ML research. Researchers affiliated with well-funded institutions or technology companies have access to private GPU clusters and cloud compute credits. Independent researchers, students, and practitioners in developing economies like India — where GPU hardware is expensive and cloud billing is in foreign currency — are systematically disadvantaged. Academic environments are little better: university high-performance computing (HPC) clusters routinely carry multi-day job queues, and free-tier cloud offerings (Google Colab, Kaggle Notebooks) impose runtime limits, random disconnections, and no guarantees of execution.

Simultaneously, there exists a significant reservoir of idle computational resources. Consumer gaming GPUs — NVIDIA GeForce RTX series, GTX series — sit underutilised during off-hours in homes, dormitories, and offices worldwide. An RTX 4090 delivers approximately 82.6 TFLOPS of FP32 throughput, compared to 14 TFLOPS for an AWS V100 instance. These machines represent enormous latent computing potential that is entirely unused for ML training tasks.

DTrain was conceived to bridge these two realities: it creates a marketplace where ML compute demand meets idle GPU supply, with all the infrastructure complexity — job management, payment, execution, output delivery — handled transparently by the platform.

1.2 Problem Statement

The central problem addressed by DTrain can be articulated across six distinct pain points that exist in the current ML training ecosystem:

Pain Point	Reality Today
Cloud GPU Costs	AWS p3.2xlarge ~Rs.280/hr. A BERT fine-tune = Rs.840–Rs.1120 for one job.

Queue Times	Colab Pro disconnects. Kaggle limits. University HPC has multi-day queues.
Infrastructure Complexity	Setting up CUDA drivers, Docker, cloud IAM, storage buckets takes hours.
No Live Feedback	Cloud batch jobs give no output until they complete or crash silently.
Wasted Idle GPUs	Millions of consumer GPUs sit at 0% utilisation every night.
No Micropayment Layer	Ad-hoc compute sharing has no billing, escrow, or trust mechanism.

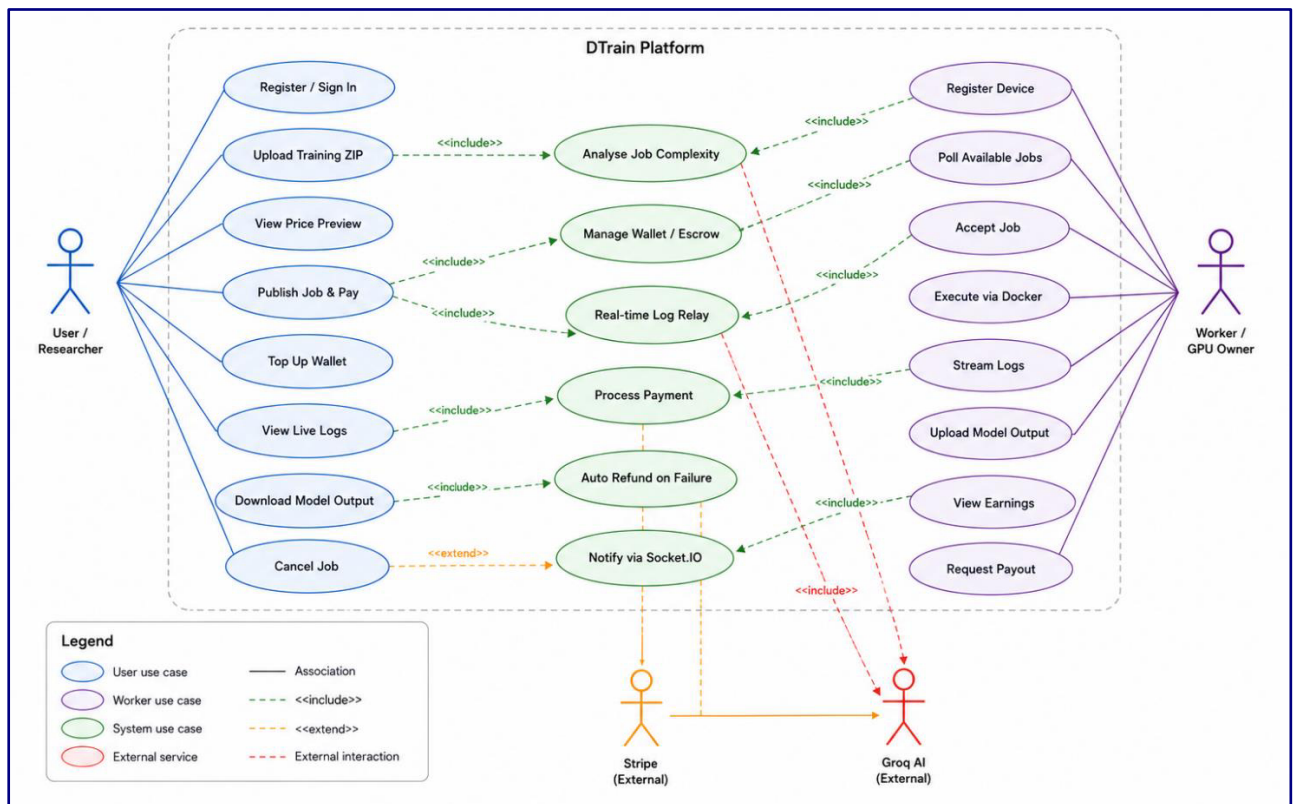


Fig 1.1 — Use Case Diagram — DTrain System Actors and Interactions

1.3 Objectives

The primary objectives of the DTrain project are:

- To design and implement a peer-to-peer distributed computing platform specifically optimised for AI model training workloads.
- To develop a novel two-layer AI pricing engine that accurately estimates job complexity without human intervention, combining static code analysis with LLM inference.

- To provide a seamless end-to-end user experience from training script upload to trained model download, with real-time feedback at every stage.
- To create a trust and payment infrastructure using established fintech primitives (Stripe escrow, reservation-based billing) that protects both job submitters and compute providers.
- To demonstrate that crowd-sourced consumer GPU compute can meaningfully accelerate ML training workloads compared to single low-end machines while remaining cheaper than commercial cloud alternatives.
- To package the worker-side experience as a one-click desktop installer (Electron) accessible to non-technical GPU owners, lowering the barrier to supply-side participation.

1.4 Scope of the Project

DTrain is scoped as a full-stack, multi-application software system consisting of:

- A user-facing web application (React/TypeScript) for job submission, monitoring, and model retrieval.
- A RESTful API backend (Node.js/Express.js v5) with Socket.IO for real-time bidirectional communication.
- A worker dashboard web application (React/TypeScript) for compute providers to monitor earnings and job activity.
- An Electron desktop agent that runs as a background process on worker machines, managing Docker containers for job execution.

The scope includes: JWT-based authentication, AI-powered flat-rate pricing, Stripe payment escrow with automated settlement, real-time log streaming via Socket.IO, Docker container lifecycle management, Supabase object storage for training artifacts, Redis-based job queue pub-sub, and MongoDB Atlas as the primary database. The scope does not include GPU pass-through to Docker, multi-node distributed training within a single job, or mobile client applications.

1.4 Report Organisation

The remainder of this report is organised as follows: Chapter 2 reviews existing literature and related work in distributed computing, cloud ML platforms, and peer-to-peer systems. Chapter 3 describes the proposed architecture and design decisions. Chapter 4 details individual modules and features. Chapter 5 documents the technology stack. Chapter 6 provides technical implementation details including database schemas, API specifications, and algorithms. Chapter 7 presents results and performance benchmarks. Chapters 8, 9, and 10 cover applications, future scope, and conclusion respectively.

REVIEW OF LITERATURE

2.1 Report Organisation

The dominant paradigm for large-scale ML training is cloud-based compute. Amazon Web Services (AWS) SageMaker, Google Cloud Vertex AI, and Microsoft Azure Machine Learning provide managed training environments with on-demand GPU access. These platforms offer scalability and reliability but at significant cost: AWS p3.2xlarge instances with NVIDIA V100 GPUs are priced at approximately USD 3.06/hr (Rs.255/hr at current exchange rates), while p4d.24xlarge instances with 8x A100 GPUs exceed USD 32/hr.

Academic work by Narayanan et al. (2021) on the efficiency of large-model training pipelines demonstrated that even well-optimised distributed training on cloud infrastructure spends 15–40% of wall-clock time on communication overhead rather than computation. DTrain's architecture circumvents this by treating each job as a single-node execution on the worker machine, eliminating inter-node communication overhead.

2.2 Peer-to-Peer and Volunteer Computing

The concept of volunteer distributed computing predates modern cloud services. SETI@home (1999) and BOINC (Berkeley Open Infrastructure for Network Computing, 2002) pioneered aggregating idle consumer CPUs for scientific computation. These systems demonstrated that enormous aggregate compute power exists in consumer hardware but lacked economic incentive mechanisms for participants.

Golem Network (2018, Ethereum-based) extended the volunteer computing model with a cryptocurrency payment layer, enabling compute providers to earn GNT tokens for completed tasks. However, cryptocurrency payments introduce friction for mainstream users and are subject to significant price volatility. DTrain adopts a fiat currency (INR) payment model via Stripe, targeting mainstream adoption without requiring participants to hold digital assets.

Vast.ai is a commercial GPU marketplace that connects GPU owners with ML researchers, allowing hourly rental of GPU capacity from a centralised pool. DTrain differs from Vast.ai in its abstraction model: rather than renting raw GPU access (which requires users to configure their own CUDA environment), DTrain accepts a self-contained Python training ZIP and manages all execution infrastructure transparently, significantly lowering the barrier for non-expert users.

2.3 Serverless and Function-as-a-Service Compute for ML

Serverless ML inference has received considerable attention (AWS Lambda, Cloudflare Workers AI), but serverless architectures impose strict memory, timeout, and CPU-only constraints that make them

unsuitable for training workloads. Modal Labs and Replicate have emerged as GPU-optimised serverless platforms that accept Python code directly, pricing by wall-clock seconds of GPU usage. These systems inspire DTrain's 'upload code, get results' UX philosophy, though DTrain differentiates with its flat-tier pricing model (eliminating per-second uncertainty) and crowd-sourced supply layer.

2.4 Docker-Based Workload Isolation

Docker container technology has become the de facto standard for portable, isolated compute workloads. Merkel (2014) introduced Docker's lightweight container model as an alternative to full virtual machine isolation. For ML training, Docker provides reproducible environments: a container started from `python:3.10-slim` with specific packages installed from `requirements.txt` produces identical results regardless of the underlying host hardware.

The Electron worker agent in DTrain leverages Docker's spawn APIs to run training containers in an isolated namespace, capturing `stdout/stderr` line by line. This approach prevents cross-job data leakage and ensures that a misbehaved training script cannot damage the worker's host filesystem.

2.5 Real-Time Log Streaming in Distributed Systems

Real-time observability of distributed job execution is an active area of development. Kubernetes-native log aggregators (Loki, Fluentd), Celery's task monitoring (Flower), and Apache Airflow's task instance logs all address the challenge of surfacing worker-generated output to operators in near real-time. Socket.IO provides a WebSocket-based abstraction well-suited to browser clients. DTrain uses Socket.IO's room-based pub-sub model: each job has a room (`job:<id>`), the worker agent POSTs log lines to the backend, and the backend relays them to all sockets that have joined that room.

2.6 LLM-Based Code Analysis

Recent work on using large language models for code understanding has demonstrated that models such as GPT-4 and Claude can accurately estimate computational complexity from code snippets. The Groq inference API (used in DTrain) provides extremely low-latency LLM inference — typically under 500ms per request — enabling real-time analysis of submitted training scripts during the job creation flow. DTrain's hybrid approach combines a deterministic rule-based scorer with LLM inference, inspired by ensemble methods in classical machine learning: the rule-based layer provides a reliable floor, while the LLM adds semantic understanding of code patterns that rule-based systems may miss.

2.7 Payment Escrow Systems for Digital Marketplaces

The economics of two-sided digital marketplaces have been extensively studied. In gig-economy platforms (Uber, Airbnb, Fiverr), escrow mechanisms protect both buyers and sellers by holding funds in a trusted third-party account until service delivery is confirmed. Stripe's Payment Intents API implements exactly this model: a payment intent can be created, authorised (funds reserved), and either captured (charged) or cancelled (released) based on downstream events. DTrain implements this escrow pattern natively: funds are reserved on job publication, charged on successful completion, and fully released on failure or cancellation.

PROPOSED WORK

3.1 System Overview

DTrain is proposed as a four-component distributed system that together implement a complete peer-to-peer ML training marketplace. The central innovation is the treatment of GPU compute as a commodity service where all complexity — pricing estimation, job isolation, payment settlement, output delivery — is abstracted away from both the buyer (ML researcher) and the seller (GPU owner).

The system is conceptually modelled after food delivery platforms: a user places an order (job submission), the platform handles vendor selection and payment (worker assignment and escrow), a delivery agent executes the task (Electron worker running Docker), and the user receives the output (trained model). This analogy drives UX design decisions throughout the platform.

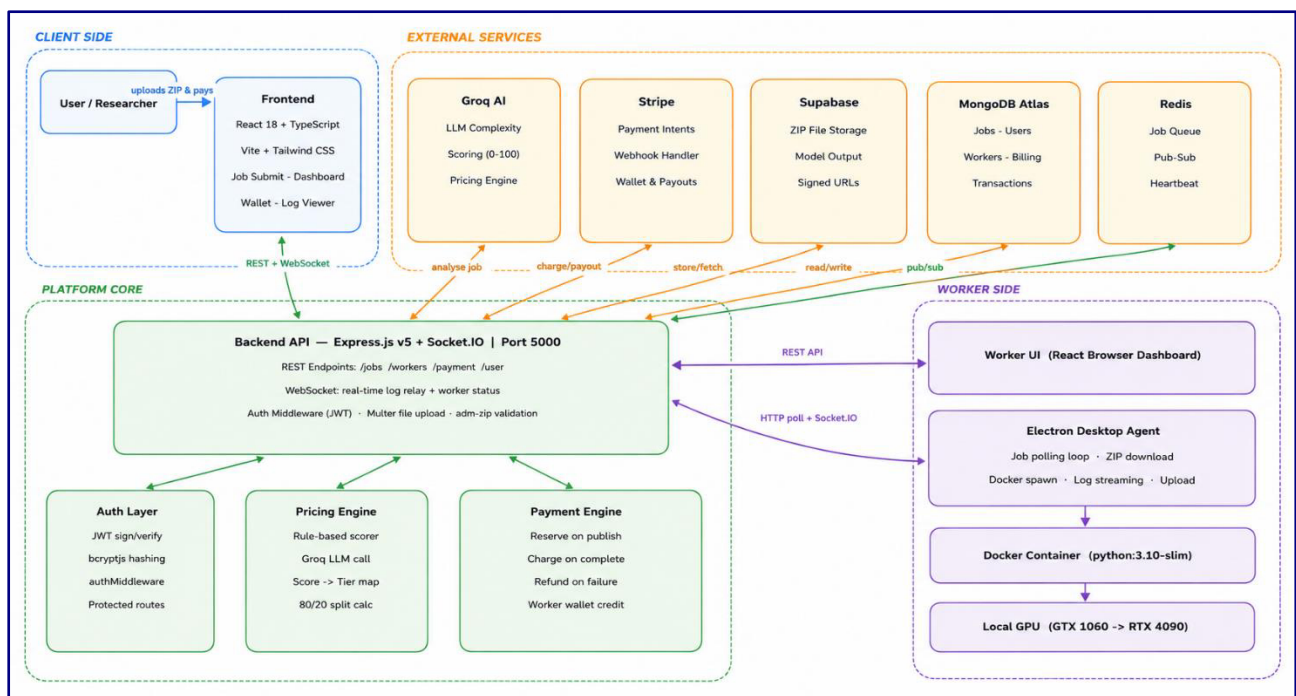


Fig 3.1 — DTrain System Architecture — Four Component Layers

3.2 Proposed Architecture

The proposed architecture separates concerns into four independent application layers, each deployable independently:

Component	Technology	Responsibility
frontend/	React 18, TypeScript, Vite, Tailwind	User dashboard — job submission, live log view, wallet top-up, model download
backend/	Express.js v5, Socket.IO, Mongoose	Central API — auth, job lifecycle, AI pricing, payment webhooks, real-time log relay
worker-ui/	React, Tailwind, Vite	Browser UI for workers — view earnings, request payouts, set preferences
electron-worker/	Electron, Node.js, Docker	Desktop agent — polls jobs, spins Docker containers, streams logs, uploads output

3.3 Job Lifecycle Design

The core design innovation in DTrain is the job state machine, which enforces clear separation between the draft phase (pricing review before payment), the pending phase (ready for worker pickup), and the execution phase (active training). This state machine prevents common race conditions in multi-worker scenarios and protects users from being charged for jobs that never execute.

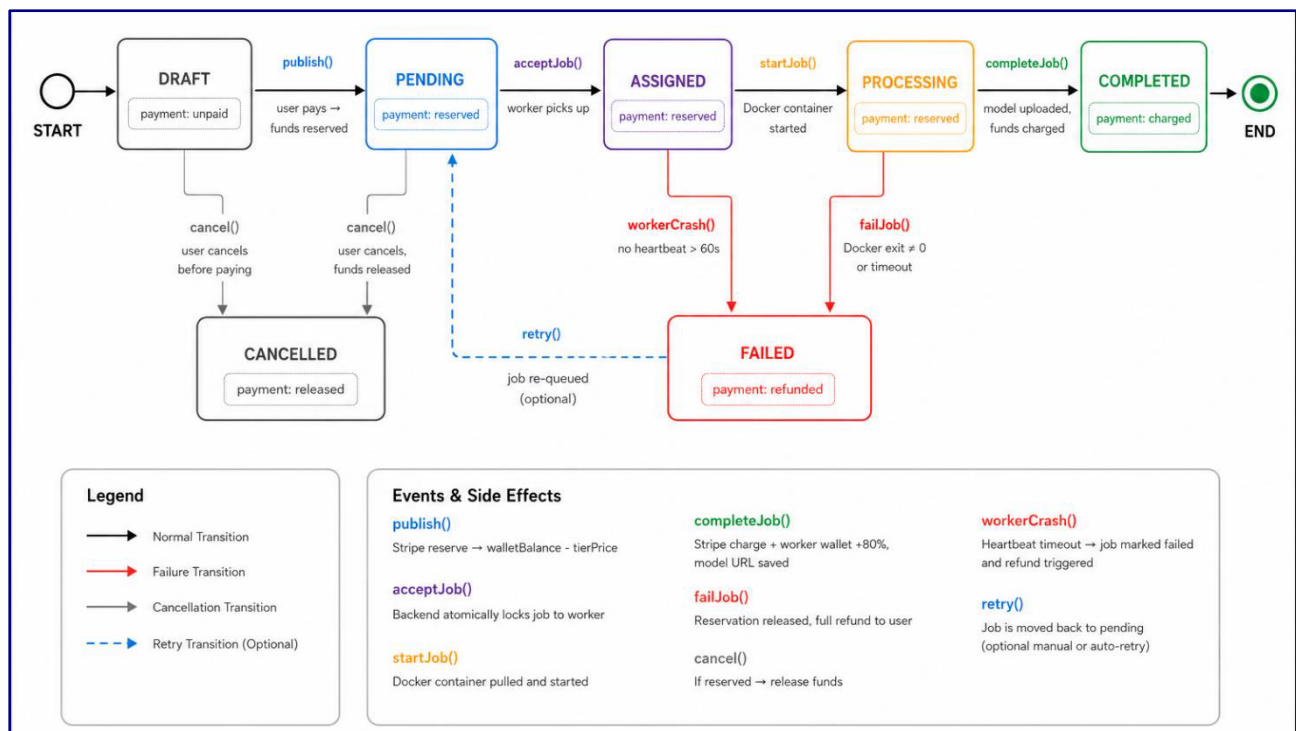


Fig 3.2 — Job Lifecycle State Machine — Status Transitions

The job status transitions are as follows:

- draft to pending: User reviews AI price and publishes the job with fund reservation. Job becomes visible to workers.
- pending to assigned: A worker atomically claims the job. Backend prevents two workers from claiming simultaneously.
- assigned to processing: Worker calls /start-job, triggering the timing clock and beginning Docker execution.
- processing to completed: Worker calls /complete-job after uploading output. Backend charges funds and credits worker wallet.
- processing to failed: Docker exits with error or worker calls /fail-job. Backend releases fund reservation (full refund to user).
- Any status to cancelled: User cancels before assignment; fund reservation is released.
- assigned to pending (timeout): If a worker accepts but does not start within 2 minutes, the job is re-queued for another worker.

3.4 AI Pricing Engine Design

The proposed two-layer pricing engine addresses the fundamental challenge of estimating fair compensation for heterogeneous ML workloads without human review. The design goals are:

- Consistency: identical jobs must always receive the same price.
- Fairness: heavy workloads must receive higher prices than trivial jobs.
- Resistance to manipulation: users cannot artificially lower prices by adding misleading comments to their code.

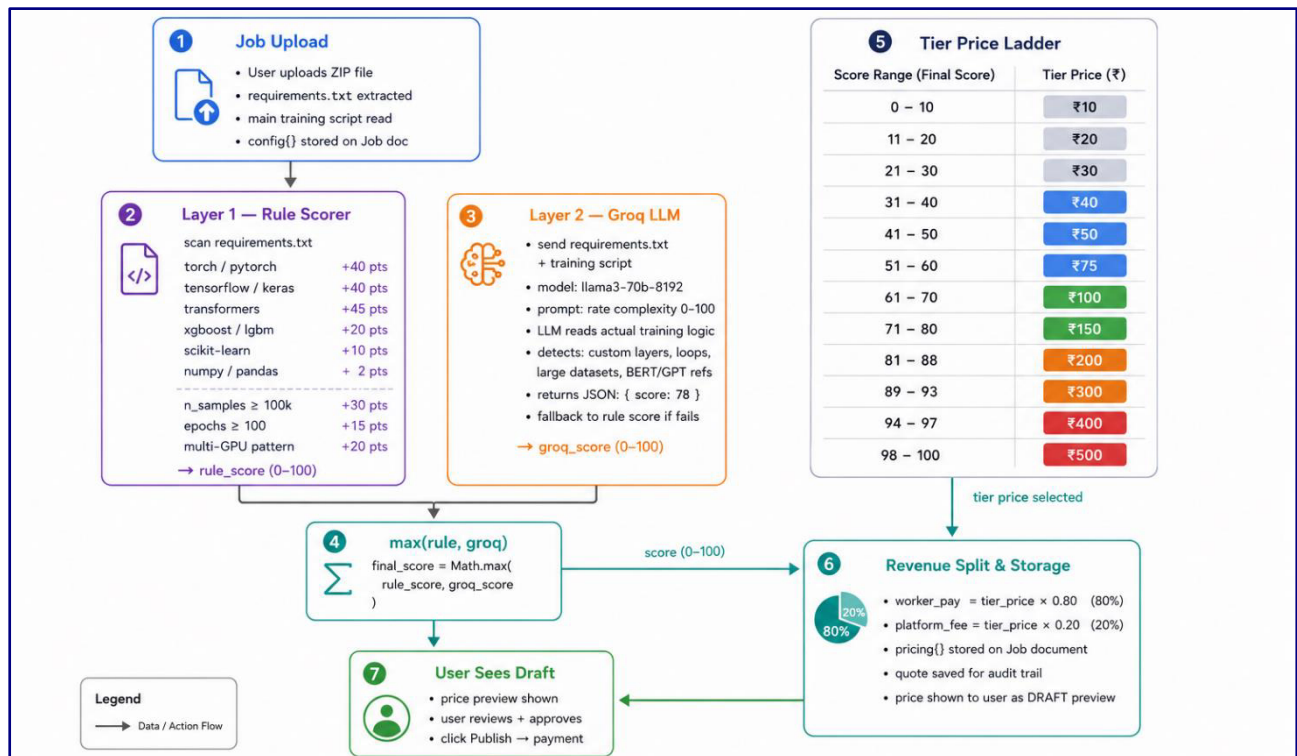


Fig 3.3 — AI Pricing Engine — Two-Layer Scoring Flow

Layer 1: Rule-Based Static Analyser: Scans both files with deterministic regex patterns. Comments are stripped from requirements.txt before matching to prevent false positives. Library weights reflect the computational cost of each framework: PyTorch/TensorFlow (+40 pts), Hugging Face Transformers/Diffusers (+45 pts), gradient boosting ensembles (+20 pts), scikit-learn (+10 pts).

Layer 2: Groq LLM: Receives the full prompt including both files (up to 3000 chars of training script) and returns a tier from the fixed ladder. The LLM's output is validated against known tier values; invalid responses fall back to the rule-based score.

Final price = $\max(\text{rule_score_tier}, \text{llm_suggested_tier})$. The rule-based layer sets a floor; the LLM can only increase the price, never decrease it below what static analysis dictates.

3.5 Payment Architecture Design

The payment system is designed around Stripe's PaymentIntent reservation model, mirroring the 'hold and capture' pattern used by hotel booking platforms. The design ensures:

- Users are never charged for jobs that do not complete successfully.
- Workers receive payment only for verified successful completions.
- The platform takes no fee on failed jobs.
- Fund reservation happens at job publication, preventing submission of jobs without sufficient balance.

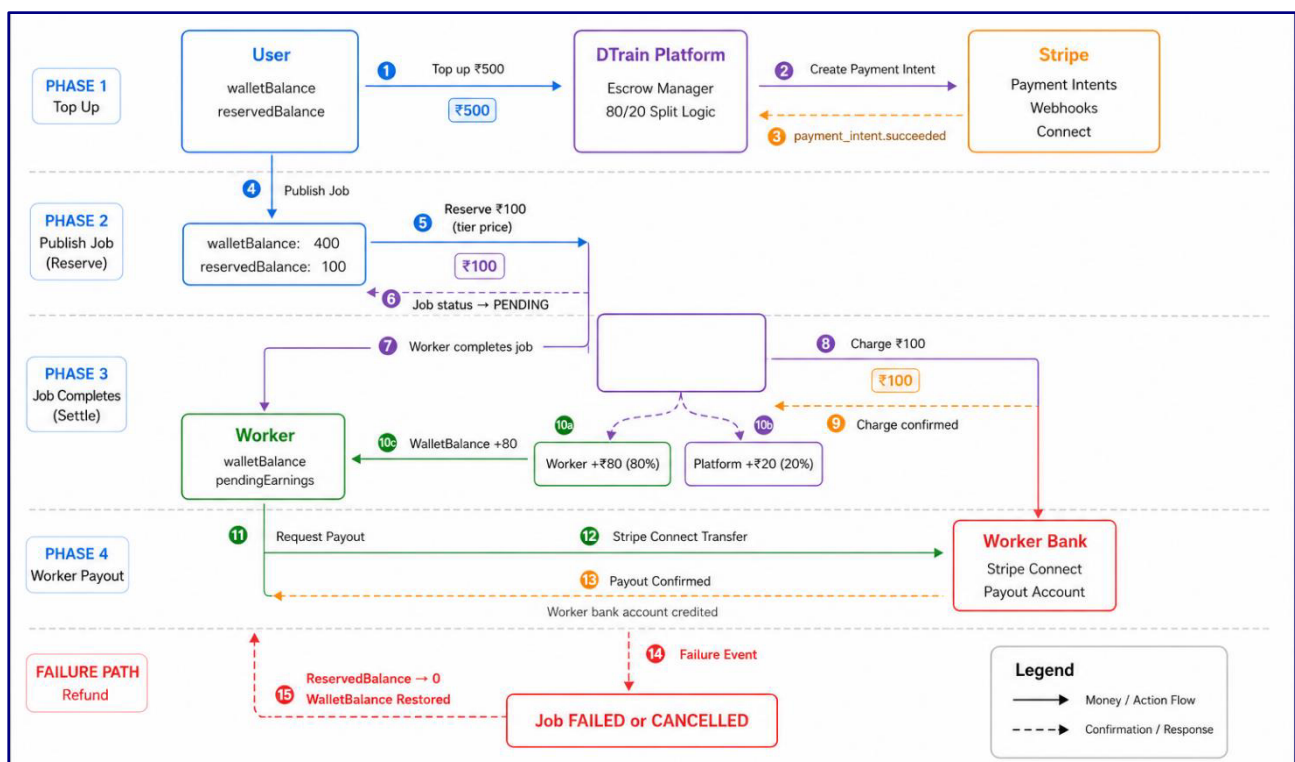


Fig 3.4 — Payment Flow — Reserve, Charge, and Payout Sequence

PROJECT MODULES & FEATURES

4.1 User Authentication Module

The authentication module provides standard email/password registration and login flows. Password hashing uses bcryptjs with a cost factor of 10, ensuring that even a database breach does not expose plaintext credentials. On successful login, the backend issues a JWT signed with a server-side secret (JWT_SECRET env variable). The token encodes the userId and an expiry claim.

Frontend components (SignIn.tsx, SignUp.tsx) store the token in localStorage under the key dtrain_token. A ProtectedRoute wrapper in App.tsx checks for token presence before rendering any authenticated views. The authMiddleware.js on the backend verifies and decodes the token on every protected API call, attaching req.user.userId for use in route handlers. Worker authentication follows the same JWT pattern but with a separate registration flow via the Electron app.

4.2 Job Submission Module

The job submission module (JobSubmission.tsx + /api/jobs/create endpoint) implements a multi-step flow:

Step	Action	Implementation Detail
1	Client-side ZIP validation	JSZip reads the uploaded file in-browser and checks for requirements.txt and the entry file before any server call.
2	Server-side ZIP validation	AdmZip re-validates entries on the server, preventing bypass of client-side checks.
3	File content extraction	requirements.txt and the entry Python file are read into memory as UTF-8 strings for the pricing engine.
4	AI pricing	assignTierWithGroq() runs the rule-based scorer and Groq LLM call. Returns a tier price in Rs.
5	Supabase upload	ZIP is uploaded to Supabase Storage bucket under jobs/ with a timestamped path. Public URL is recorded.
6	Draft job creation	Job document saved to MongoDB with status: draft, paymentStatus: unpaid. Not visible to workers yet.

7	Price review	Frontend shows the AI-assigned tier price. User reviews and decides before committing to payment.
---	--------------	---

4.3 Job Publishing and Fund Reservation Module

When a user clicks 'Publish & Pay' after reviewing the draft price, the frontend calls POST `/api/jobs/:id/publish`. This endpoint executes atomically:

1. Validates the user has sufficient available wallet balance (`walletBalance` minus `reservedBalance` $\geq$ `tierPrice`).
2. Increments `reservedBalance` by `tierPrice` to lock the funds against spending elsewhere.
3. Updates job status to 'pending' and `paymentStatus` to 'reserved'.
4. Emits a Socket.IO event to all connected clients indicating a new job is available in the pool.

This reservation-first model ensures the platform can always honour payment commitments without risk of overdraft scenarios.

4.4 Worker Agent Module (Electron Desktop Application)

The Electron desktop agent (`electron-worker/main.js`) is the most complex component in the DTrain system. It runs as a desktop application on the worker machine, embedding a Chromium-based renderer that loads the worker-ui React application. The main process handles all backend communication and Docker management via Electron's IPC (Inter-Process Communication) bridge.

4.4.1 Hardware Detection

On first launch, the agent calls `ipcMain.handle('get-device-info')` which uses Node.js's built-in `os` module to detect CPU model and core count, total RAM, and operating system. GPU detection is platform-specific: Windows uses the `wmic path win32_VideoController get name` command, Linux uses `lspci | grep -i vga`, and macOS uses `system_profiler SPDisplaysDataType`. The results are sent to the backend on worker registration.

4.4.2 Job Polling and Heartbeat

The agent polls `GET /api/worker/available-jobs?deviceId=...` every 5 seconds. This serves dual purpose: fetching new available jobs and acting as a heartbeat to the backend. If the backend receives no poll for 60 seconds, it marks the worker as offline and emits a `worker_status_changed` Socket.IO event. The backend also runs a secondary heartbeat checker every 30 seconds that marks any workers with `lastHeartbeatAt` older than 60s as offline — a safety net for hard crashes that do not produce a clean TCP disconnect.

4.4.3 Docker Job Execution

When a job is accepted, the `run-test-job` IPC handler performs the following sequence:

- Calls `POST /api/worker/start-job` to notify the backend and trigger the job timing clock.
- Downloads the training ZIP from the Supabase URL stored in the job document.

- Extracts the ZIP to a temporary directory (`os.tmpdir()/dtrain-jobs/job-<id>/`).
- Spawns a Docker container: `docker run --rm -v /tmp/job_xyz:/workspace python:3.10-slim bash -c 'pip install -r requirements.txt && python train.py'`
- Streams every line of stdout and stderr through `streamLogToBackend()`, which POSTs to `/api/worker/push-log`.
- On Docker exit code 0: zips the output directory, uploads to Supabase, and calls `/api/worker/complete-job`.
- On non-zero exit code: calls `/api/worker/fail-job` with the collected error message.

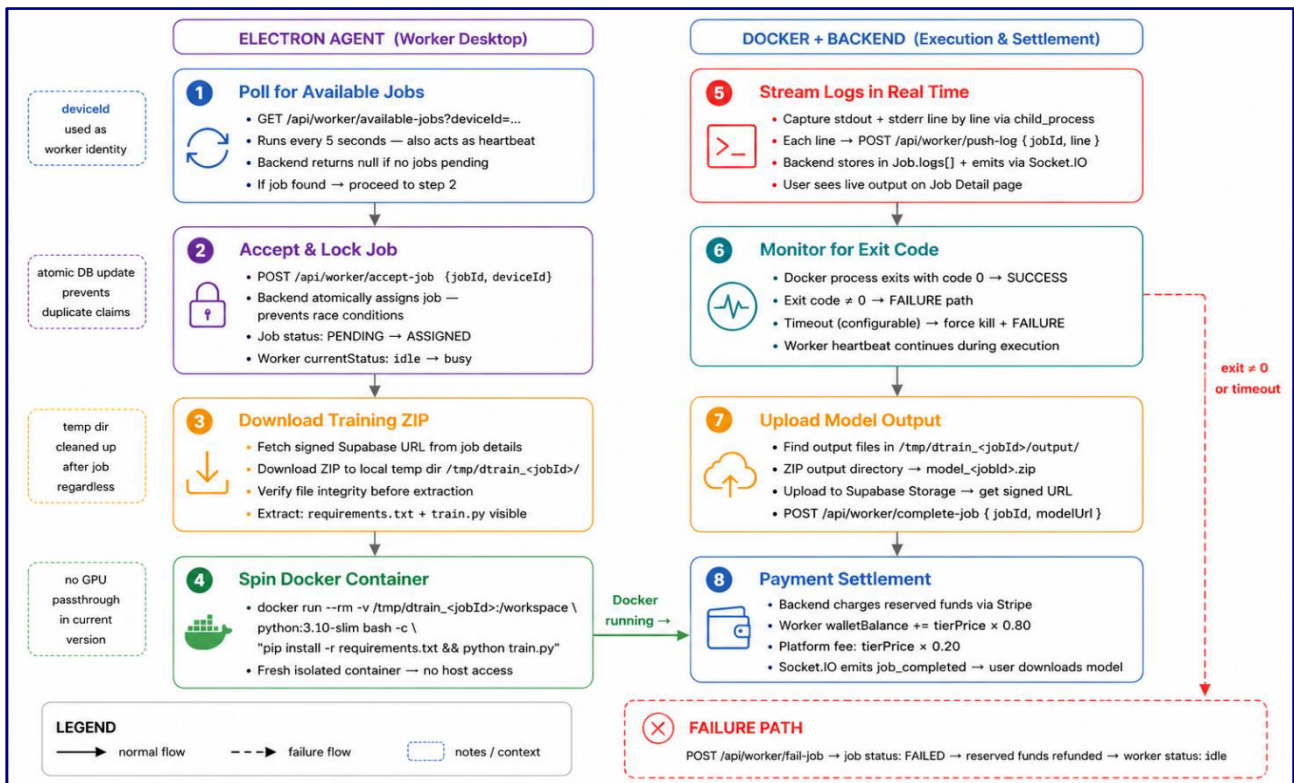


Fig 4.1 — Docker Job Execution Flow — Electron Agent to Container to Backend

4.4.4 Live Docker Metrics

While a container is running, the agent collects Docker stats (CPU%, memory usage) via `docker stats --no-stream` commands executed every 5 seconds. These metrics are POSTed to `/api/worker/metrics` and stored in the `BillingSchema` document. The frontend's `JobDetail` component displays these as real-time line charts using the `Recharts` library, giving users visibility into resource consumption during training.

4.4 Job Lifecycle Design

Log streaming connects the Docker container running on a worker's machine to the user's browser in near real-time. The complete pipeline is:

Docker stdout/stderr → Node.js child_process.spawn (Electron main process) → POST /api/worker/push-log → Express route handler → io.to('job:<id>').emit('job:log', {line}) → Socket.IO client in browser → React state update → terminal-style log view in JobDetail.tsx.

The /push-log endpoint appends the line to the job's logs array in MongoDB (enabling log persistence) and simultaneously emits the Socket.IO event. Users who join a job detail page mid-execution see all historical logs immediately from MongoDB, then receive live updates via the socket connection.

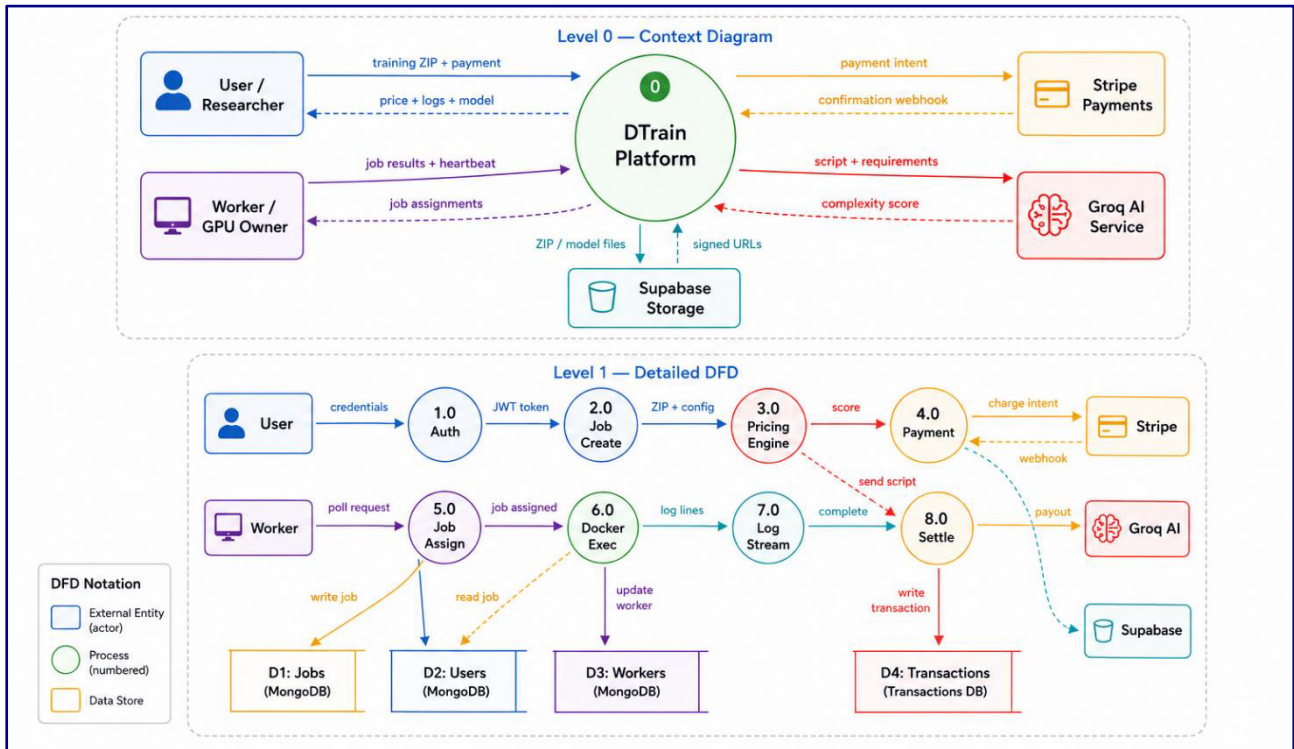


Fig 4.2 — Real-Time Data Flow Diagram — Log Streaming and Status Updates

4.6 Wallet and Payment Module

The wallet module implements a ledger-based balance system with two sub-balances stored per user:

- **walletBalance**: Total funds ever deposited. Only decreases on actual job charges.
- **reservedBalance**: Funds locked for pending or active jobs. Cannot be spent on new jobs.
- **availableBalance** (computed, not stored): walletBalance minus reservedBalance. This is the actual spendable amount.

Top-ups are processed via Stripe Embedded Checkout — a Stripe-hosted payment form embedded in a modal within the DTrain frontend. This approach delegates PCI-DSS compliance entirely to Stripe; DTrain never handles raw card data. On successful payment, Stripe sends a webhook (checkout.session.completed) to the backend, which calls addFundsToWallet() to credit the user's walletBalance.

Worker payouts use Stripe Connect: workers onboard by linking their bank account to a Stripe Connect account. The platform then initiates payouts to the connected account when the worker requests a withdrawal from their in-platform wallet.

4.7 Active Workers Dashboard

The ActiveWorkers component in the frontend provides a live view of all registered worker nodes and their current statuses. Worker status (online, idle, busy, offline) is updated in real-time via Socket.IO `worker_status_changed` events. The component displays each worker's hardware specifications (GPU model, CPU, RAM), the number of jobs completed, and current activity — providing ML researchers with visibility into the state of the compute pool before submitting a job.

4.8 Worker Registration and Onboarding

Worker onboarding through the Electron app is designed for non-technical GPU owners. On first launch, the app auto-detects hardware specifications and presents a registration form. Upon registration, the worker receives a `deviceId` that is persisted to disk (`app.getPath('userData')/device-config.json`), ensuring the same identity is used across app restarts. The Worker Dashboard (`worker-ui`) then provides the primary interface for monitoring pending jobs, managing the payout wallet, and configuring pricing preferences.

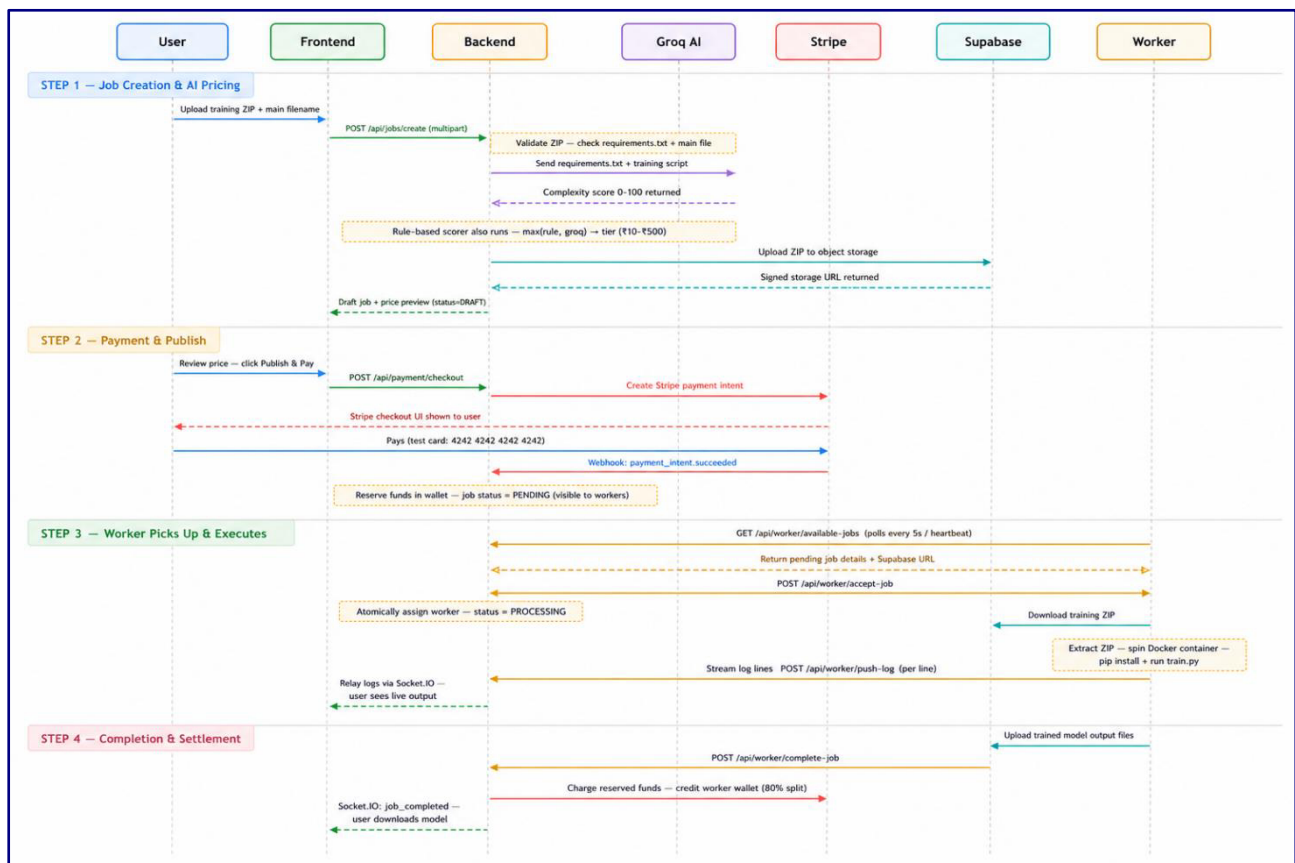


Fig 4.3 — Sequence Diagram — Full Job Lifecycle from User to Worker

4.9 In-App Documentation Module

Both the user dashboard (`frontend/`) and the worker dashboard (`worker-ui/`) include a Documentation component that renders a comprehensive in-app guide. The user documentation covers: how to

prepare a training ZIP, entry file conventions, what requirements.txt entries affect pricing, how to download the trained model, and troubleshooting common errors. The worker documentation explains: system requirements, Docker installation steps, how the wallet and payout system works, and how to optimise the machine for job execution.

4.10 Assigned Job Timeout and Re-queuing

A critical reliability feature is the assigned job timeout checker. If a worker accepts a job (status: assigned) but fails to call /start-job within 2 minutes — indicating the worker may have crashed between accept and start — the backend automatically resets the job to 'pending' status, clears the assignedWorkerId and startTime fields, and emits a Socket.IO event so the job becomes available to other workers. This prevents jobs from being permanently locked to unresponsive workers.

PLATFORM & TECHNOLOGY

5.1 Backend Technology Stack

The backend is a Node.js v20+ application using Express.js v5 — the first major update to Express since v4 was released in 2014. Express v5 introduces automatic async error handling: unhandled Promise rejections in route handlers are forwarded to Express error middleware automatically, reducing boilerplate for robust async routes. The server runs on port 5000 and exposes REST endpoints under four route namespaces.

Technology	Version	Role in DTrain
Node.js	v20+	JavaScript runtime — async I/O, child_process for Docker management
Express.js	v5	REST API framework — routing, middleware pipeline, async error handling
Socket.IO	4.8	Real-time bidirectional communication — log streaming, worker/job status events
MongoDB + Mongoose	Atlas + v9	Primary database — jobs, users, workers, billing, transactions
Redis (ioredis)	v5	Job queue pub-sub messaging between backend and worker nodes
Groq SDK	v1.2	Ultra-low-latency LLM inference for the AI pricing engine
Stripe SDK	v22	Payment intents, webhooks, wallet management, Connect payouts
Supabase	v2	S3-compatible object storage for ZIP uploads and model outputs (Mumbai region)
JWT (jsonwebtoken)	v9	Stateless authentication — access tokens signed with JWT_SECRET
bcryptjs	v3	Password hashing with salt rounds (cost factor 10)

Multer	v2	Multipart form data handling for ZIP file uploads in-memory
AdmZip	v0.5	In-memory ZIP reading and file extraction on the server
Morgan	v1.10	HTTP request logging in development mode

5.2 Frontend Technology Stack (User Dashboard)

The user-facing frontend is a React 18.3 single-page application (SPA) built with Vite 5.4 and TypeScript 5.5. Tailwind CSS 3.4 provides the utility-first styling framework. The visual design follows a neobrutalist aesthetic: thick black borders (`border-[3px] border-slate-900`), bold shadow offsets, and a warm cream/orange background palette (`#FFEFE1`).

Library	Version	Purpose
React	18.3	UI component framework with hooks for state management
TypeScript	5.5	Static typing — catches interface mismatches at compile time
Vite	5.4	Dev server and build tool — Hot Module Replacement, fast cold starts
Tailwind CSS	3.4	Utility-first CSS framework for rapid UI development
Socket.IO Client	4.8	WebSocket connection for real-time log streaming and job status events
Stripe React / Stripe.js	6.6 / 9.7	Embedded Stripe Checkout modal for wallet top-up payments
Recharts	3.1	Line and bar charts for job stats and live Docker metrics
Framer Motion	12	Declarative animations and page transition effects
React Router	7.13	Client-side SPA routing — <code>/dashboard</code> , <code>/submit</code> , <code>/jobs/:id</code> , <code>/wallet</code>
React Dropzone	14	Drag-and-drop file upload zone for training ZIP files

JSZip	latest	Client-side ZIP inspection for pre-flight entry file validation
Lucide React	0.344	Icon set used throughout the UI for consistent iconography

5.3 Worker UI Technology Stack

The worker dashboard (worker-ui/) shares the same core stack as the user frontend: React 18, TypeScript 5.5, Vite 5.4, and Tailwind CSS 3.4. Socket.IO Client 4.8 is included for real-time job status updates. The worker UI is compiled into a dist/ folder that the Electron main process loads directly (mainWindow.loadFile('../worker-ui/dist/index.html')), making it the renderer process of the Electron application. The Electron contextBridge (preload.js) exposes a safe set of IPC functions to the renderer without granting full Node.js access.

5.4 Electron Worker Agent Technology

Technology	Role
Electron	Desktop app shell — BrowserWindow, IPC main/renderer bridge, system tray integration
Node.js child_process	Spawn Docker process, capture stdout/stderr line-by-line in real time via spawn()
Node.js os module	Detect CPU model, core count, total RAM, and platform identifier
Docker	Container runtime for isolated, reproducible training job execution on worker machine
node-fetch	HTTP calls from main process to backend API — job polling, log push, completion reporting
AdmZip	Extract downloaded training ZIP before mounting into Docker container workspace
electron-builder	Packages the entire app into a distributable Windows installer (.exe)

5.5 Data Storage Architecture

DTrain uses three distinct storage systems, each chosen for its specific characteristics:

5.5.1 MongoDB Atlas (Primary Database)

MongoDB's document model is a natural fit for DTrain's heterogeneous data. Job documents have a variable logs array that grows during execution; worker documents contain nested systemInfo objects with platform-specific fields; billing documents reference jobs, users, and workers. The schema-

flexibility of MongoDB allows these structures without the overhead of relational table design. The dtrain database contains six collections: users, jobs, workers, billings, transactions, and jobmetrics. MongoDB Atlas is used in the Mumbai (ap-south-1) region for low latency from India.

5.5.2 Redis (Job Queue)

Redis is used as a pub-sub message broker. When a job transitions to 'pending', the backend publishes a message to a Redis channel that worker nodes can consume. Redis also provides fast in-memory lookups for the heartbeat monitoring system without adding polling load to MongoDB. The ioredis v5 client is used for its Promise-based API and connection management features.

5.5.3 Supabase Storage (Object Storage)

Supabase provides an S3-compatible object storage layer hosted in the Mumbai (ap-south-1) region to minimise latency for Indian users. Two storage buckets are used: 'jobs' (stores user-uploaded training ZIPs) and 'models' (stores worker-uploaded output ZIPs after training completes). The Supabase client uses a service-role key for unrestricted backend access, with a 3600-second CDN cache on uploaded files for fast repeated downloads.

5.6 Network Architecture

The network topology of a deployed DTrain instance involves the following communication paths:

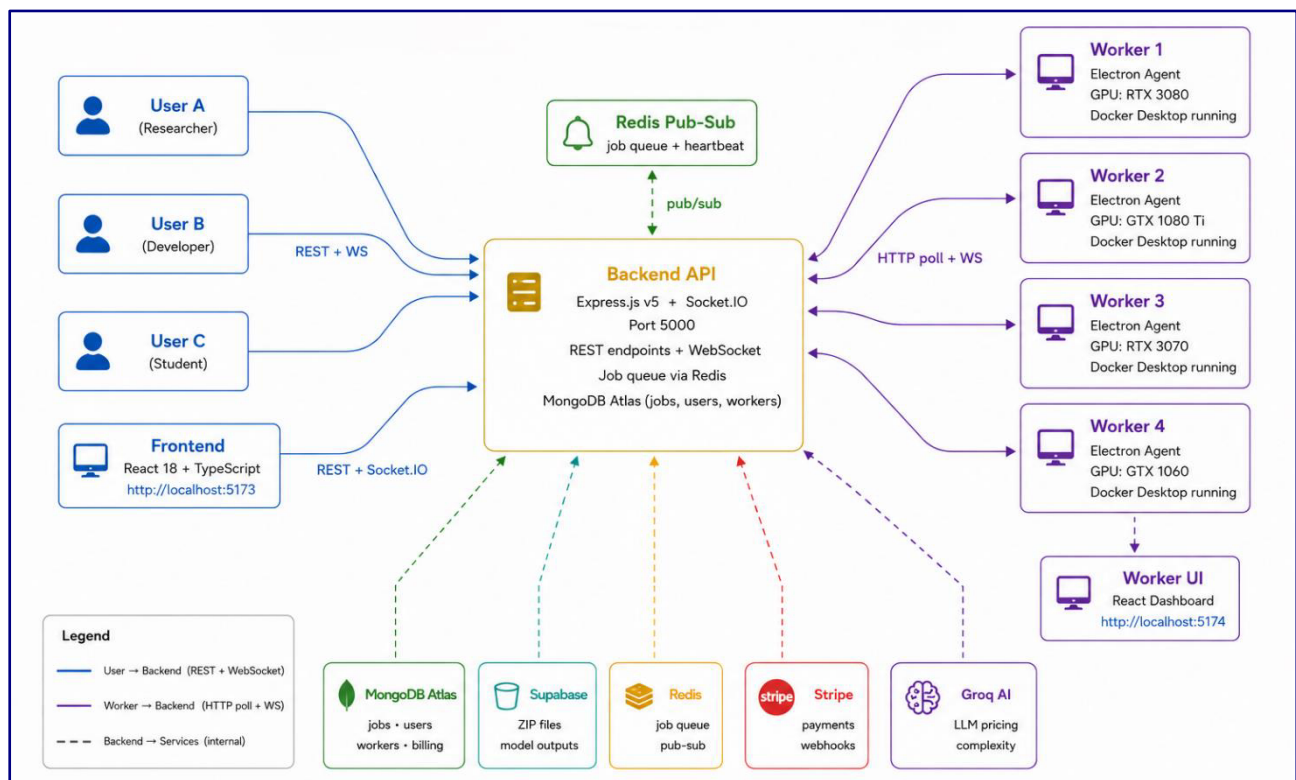


Fig 5.1 — DTrain Network Topology — Component Communication Paths

- User Browser to Backend: HTTPS REST for job management and wallet operations; WebSocket (Socket.IO) for real-time log streaming.
- Worker Browser (inside Electron) to Backend: HTTPS REST and WebSocket via Socket.IO for status updates.

- Electron Main Process to Backend: HTTP REST — job polling every 5 seconds, log push per line, completion/failure reports.
- Backend to Supabase: HTTPS via Supabase SDK for file upload/download URL generation.
- Backend to MongoDB Atlas: MongoDB Wire Protocol over TLS.
- Backend to Redis: Redis binary protocol over TCP.
- Backend to Groq API: HTTPS for LLM inference requests during job pricing.
- Backend to Stripe API: HTTPS for payment intents, webhook reception, and Connect payouts.
- Electron Worker to Supabase: HTTPS for direct large-file ZIP download and model output upload.

TECHNICAL DETAILS

6.1 Database Schema Design

The MongoDB database schema for DTrain consists of six Mongoose models. The schema design emphasises referential integrity via ObjectId references, status enums for state machine enforcement, and embedded sub-documents for related data that is always accessed together.

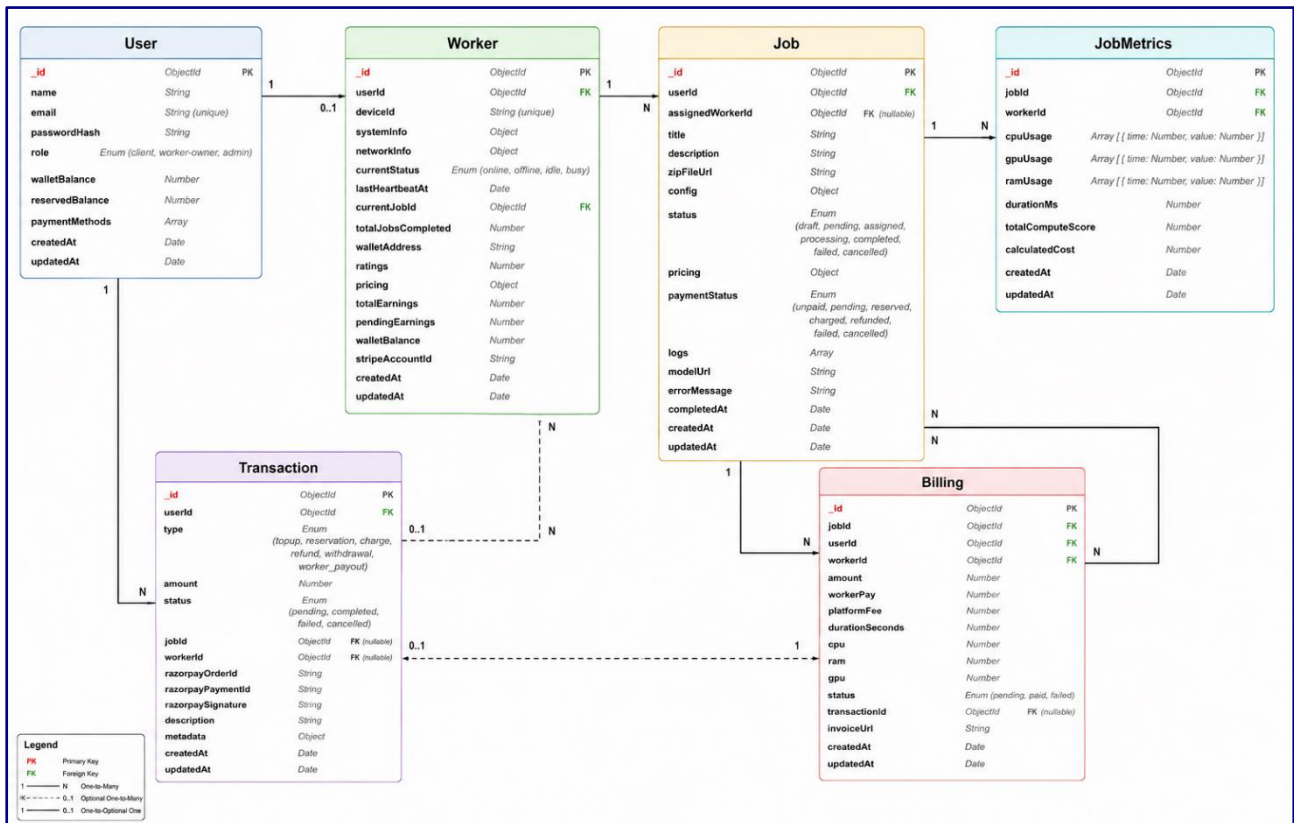


Fig 6.1 — Entity-Relationship Diagram — MongoDB Collections

6.1.1 Job Schema (JobSchema.js)

The Job schema is the central document in DTrain's data model. It captures the complete lifecycle of a training job from creation through completion.

Field	Type	Description
userId	ObjectId → User	Foreign key reference to the submitting user
status	String (enum)	State machine: draft pending assigned processing completed failed cancelled

paymentStatus	String (enum)	unpaid pending reserved charged refunded failed cancelled
zipFileUrl	String	Supabase public URL for the uploaded training ZIP file
pricing.tierPrice	Number	Flat price in INR assigned by the AI pricing engine
pricing.workerPay	Number	tierPrice x 0.80 — credited to worker on successful completion
pricing.platformFee	Number	tierPrice x 0.20 — retained by platform as service fee
pricing.startTime	Date	Set when worker calls /start-job; used for elapsed time display
logs	Array<String>	Append-only log array; each element is one line of Docker stdout/stderr
config.entryFile	String	The Python entry filename (e.g., train.py) specified by the submitting user
modelUrl	String	Supabase URL for the trained model output ZIP, set on job completion
assignedWorkerId	String	deviceId of the worker that has claimed this job
acceptedAt	Date	Timestamp when worker accepted; used by idle timeout checker (2-min threshold)

6.1.2 User Schema (UserSchema.js)

The User schema stores authentication credentials and wallet state. The dual-balance design (walletBalance + reservedBalance) is the key architectural decision: it allows the platform to compute truly spendable funds without an additional database query per job submission. The availableBalance = walletBalance - reservedBalance formula is a pure, deterministic function of these two stored fields, computed in the getAvailableBalance() helper function.

6.1.3 Worker Schema (WorkerSchema.js)

The Worker schema captures hardware specifications (systemInfo.cpu, systemInfo.gpu, systemInfo.ram), current operational status (online/offline/idle/busy), and earnings tracking (totalEarnings, pendingEarnings, walletBalance). The lastHeartbeatAt field is updated by every polling call from the Electron agent, enabling the backend's heartbeat timeout checker to detect crashed or disconnected workers within 60 seconds.

6.1.4 Transaction Schema (TransactionSchema.js)

The Transaction schema provides a complete audit ledger of all financial events. Transaction types include: topup (wallet top-up via Stripe Checkout), reservation (fund reservation on job publication), charge (deduction on job completion), refund (return on failure), withdrawal (user withdrawal), and worker_payout (payout to worker via Stripe Connect). Every financial operation creates a Transaction record, enabling full reconciliation. Indexes are created on `userId+createdAt` and `razorpayOrderId` for fast query performance.

6.1.5 Billing Schema (BillingSchema.js)

The Billing schema creates a settlement record linking a specific job to its payment outcome. It records the full tier price, the worker's 80% share, and the platform's 20% fee, along with the actual duration in seconds and Docker resource stats (cpu%, ram) captured during execution. This schema serves both financial reconciliation and post-job performance analytics.

6.1.6 JobMetrics Schema (JobMetricsSchema.js)

The JobMetrics schema stores periodic resource usage snapshots captured during job execution: training duration milestones, CPU percentage, memory usage, and GPU utilisation if available. These snapshots power the real-time and historical charts in the job detail view.\

6.2 API Specification

6.2.1 User Routes (/api/user)

Method	Endpoint	Description
POST	/signup	Register new user. Validates email uniqueness, hashes password, returns JWT on success.
POST	/signin	Authenticate user with email/password. Returns JWT access token.
GET	/profile	Returns authenticated user's details and current wallet balance. Auth required.

6.2.2 Job Routes (/api/jobs)

Method	Endpoint	Description
GET	/	List all jobs for the authenticated user (includes drafts). Sorted by <code>createdAt</code> desc.

POST	/create	Multipart: upload ZIP, validate contents, run AI pricing, save as draft. Returns tier price.
POST	/:id/publish	Reserve funds and set status to pending. Job becomes visible to workers.
POST	/:id/cancel	Cancel a draft or pending job. Releases any existing reservation.
GET	/:id	Fetch a single job document including all logs, pricing, and model URL.

6.2.3 Worker Routes (/api/worker)

Method	Endpoint	Description
POST	/register	Register a worker device with hardware specs. Upserts Worker document.
GET	/available-jobs	List pending jobs for workers. Also acts as heartbeat: updates lastHeartbeatAt.
POST	/accept-job	Atomically assign job to this worker. Prevents race condition with concurrent accepts.
POST	/start-job	Mark job as processing. Sets pricing.startTime to current timestamp.
POST	/push-log	Append a log line to job document and emit Socket.IO job:log event to browsers.
POST	/metrics	Record Docker CPU/RAM snapshot for the billing record and metrics charts.
POST	/complete-job	Mark job completed, set modelUrl. Charge user wallet, credit worker 80%.
POST	/fail-job	Mark job failed with error message. Release fund reservation for full refund.

GET	/	List all registered workers with their current status. Auth required.
-----	---	---

6.2.4 Payment Routes (/api/payment)

Method	Endpoint	Description
GET	/wallet/balance	Returns total, reserved, and available balance for authenticated user.
POST	/wallet/topup	Create Stripe Embedded Checkout session. Minimum Rs.50 (Stripe minimum).
POST	/webhook	Stripe webhook endpoint. Processes checkout.session.completed events to credit wallet.
POST	/worker/payout	Initiate Stripe Connect payout to worker's linked bank account.

6.3 The AI Pricing Algorithm in Detail

The complete pricing algorithm is implemented in backend/utils/paymentHelpers.js. The function assignTierWithGroq() is the single entry point, called synchronously during job creation. It operates in two sequential phases:

6.3.1 Phase 1 — Rule-Based Scoring (scoreJobComplexity)

The rule-based scorer processes two text inputs: the requirements.txt content and the main training script. Before matching, comment lines are stripped from requirements.txt using a split-filter-join pipeline. This is a deliberate anti-gaming measure: a comment like '# torch for heavy training' would otherwise trigger the PyTorch scoring rule.

Detection Pattern	Points	Rationale
torch or pytorch in requirements.txt	+40	PyTorch training is GPU-intensive; even CPU training is slow
tensorflow or keras in requirements.txt	+40	TensorFlow graph execution comparable in cost to PyTorch

transformers or diffusers in requirements.txt	+45	Hugging Face models are large; fine-tuning is extremely GPU-heavy
xgboost, lightgbm, or catboost in requirements.txt	+20	Gradient boosting: parallelisable but deep trees add significant time
scikit-learn or sklearn in requirements.txt	+10	Traditional ML on CPU — moderate but non-trivial compute
numpy or pandas in requirements.txt	+2	Data loading utilities; low direct compute overhead
n_samples >= 100,000 detected in code	+30	Very large dataset — long training loop regardless of model type
n_samples >= 50,000 detected in code	+20	Medium dataset — significantly longer than small training sets
GridSearchCV or RandomizedSearchCV in code	+25	Hyperparameter search multiplies training time by n_iter x cv_folds
epochs >= 50 in code	+20	Long training runs: many passes through the dataset
nn.Module, Conv2d, LSTM, or Transformer in code	+35	Neural network layer definitions — deep learning execution path
Multiple model types (>= 4 classifiers/regressors)	+15	Ensemble or comparison scripts train multiple models in sequence

6.3.2 Phase 2 — Groq LLM Estimation

The Groq LLM receives a structured prompt containing: (a) the complete tier ladder, (b) a rule set for tier selection ordered by precedence, and (c) up to 3000 characters each of requirements.txt and the training script. The LLM is instructed to return only a single number from the tier ladder (10, 20, 30, 40, 50, 75, 100, 150, 200, 300, 400, or 500). If the response cannot be parsed or falls outside the known tier values, the system falls back to the rule-based score.

6.3.3 Final Score and Revenue Split

The final tier price is the maximum of the rule-based tier floor and the Groq LLM suggestion. The 80/20 revenue split is then applied:

Formula	Example (Rs.200 job)	Note
Worker Pay = tierPrice x 0.80	Rs.200 x 0.80 = Rs.160	Credited to worker wallet on job completion
Platform Fee = tierPrice x 0.20	Rs.200 x 0.20 = Rs.40	Retained by platform as marketplace commission

6.4 Socket.IO Event Reference

Event Name	Direction	Payload and Purpose
register_worker	C to S	Worker identifies itself on connect: { deviceId }. Backend joins socket to worker:<id> room.
join_job	C to S	Browser joins job room: { jobId }. Enables targeted log delivery without global broadcast.
leave_job	C to S	Browser leaves job room when navigating away from job detail view.
job:log	S to C	One log line from Docker: { jobId, line, timestamp }. Emitted per /push-log call.
job:time_update	S to C	Every 1 second for active jobs: { jobId, elapsedSeconds, tierPrice }.
job_status_changed	S to C (broadcast)	Job state machine transition: { jobId, status, reason, timestamp }.
worker_status_changed	S to C (broadcast)	Worker online/offline state change: { workerId, deviceId, status, timestamp }.

6.5 Background Interval Tasks

6.5.1 Real-Time Time Tracking (1-second interval)

Queries all jobs with status 'assigned' or 'processing' that have a pricing.startTime. For each active job, emits a job:time_update Socket.IO event with elapsed seconds since startTime. This drives the live timer display in the user's job detail view. This is display-only; it does not affect billing, which is a fixed flat fee set at job creation.

6.5.2 Heartbeat Timeout Checker (30-second interval)

Queries all workers with `currentStatus` `online/idle/busy` that have `lastHeartbeatAt` older than 60 seconds. Marks them offline and emits `worker_status_changed` events to all connected browsers. This handles the case where a worker machine loses network connectivity without cleanly closing its `Socket.IO` connection.

6.5.3 Assigned Job Idle Timeout Checker (30-second interval)

Queries all jobs with status `'assigned'` where `acceptedAt` is older than 2 minutes. Re-queues these jobs to `'pending'` status, clearing the `assignedWorkerId`, `acceptedAt`, and `pricing.startTime` fields. Emits `job_status_changed` events so connected browsers update their job list. This prevents permanent job locks when a worker crashes between the `accept` and `start` steps.

6.6 Security Implementation

DTrain implements several security measures appropriate for a beta-stage marketplace platform:

- **JWT Authentication:** All user-facing API endpoints require a valid JWT in the `Authorization: Bearer` header. Tokens are validated on every request by `authMiddleware.js`. Worker endpoints use the same JWT mechanism for authenticated operations.
- **Password Security:** `bcryptjs` with cost factor 10 ensures that password hashing takes approximately 100ms — fast enough for user experience but slow enough to make brute-force attacks impractical.
- **Client-side and Server-side ZIP Validation:** Double validation (`JSZip` in browser + `AdmZip` on server) prevents malformed or malicious ZIP files from reaching the storage layer.
- **Stripe Webhook Signature Verification:** The `/webhook` endpoint verifies the `Stripe-Signature` header using the webhook secret before processing any payment events. This prevents fake webhook injection attacks.
- **Docker Isolation:** Each training job runs in a sandboxed Docker container with `--rm` flag (auto-removal after exit) and volume mounting limited to the extracted job directory. The container cannot access the host network or other filesystem paths.
- **CORS Configuration:** The backend uses `cors({ origin: '*', credentials: true })` for development convenience. Production deployment restrict origins to known frontend domains.

SYSTEM IMPLEMENTATION & UI

This chapter presents the DTrain platform as a fully implemented and operational system, walking through the application screen by screen across both the user-facing web dashboard and the worker-side Electron desktop agent. The user-facing frontend follows a neobrutalist design language — thick black borders, bold drop shadows, and a warm cream background (#FFEFEE1) with deep orange accents — giving the platform a distinct visual identity. The worker-facing interface, rendered inside the Electron desktop application, follows a clean complementary theme optimised for a desktop viewport. The walkthrough below follows the natural journey of both actors: an ML researcher from landing page through to downloading a trained model, and a GPU owner from device registration through to receiving payment for a completed job.

7.1 Landing Page and Authentication

The DTrain landing page (Figure 7.1) is the entry point for all visitors — both ML researchers and GPU owners — and is the only page accessible without authentication. The hero section features the platform tagline 'Distributed AI Model Training — Powered by the Crowd' in large bold type against the warm cream background. Three feature cards below the headline communicate the platform's core differentiators at a glance: Decentralised Training (leveraging idle consumer GPUs), Secure Execution (Docker-isolated containers that prevent cross-job data leakage), and Real-Time Monitoring (live training logs streamed directly to the browser). A 'Get Started' call-to-action leads new users into registration, while a 'View Docs' link opens the in-app documentation. Every other route in the application is guarded by a ProtectedRoute component that checks for a valid JWT token in localStorage, silently redirecting unauthenticated visitors back to this page.

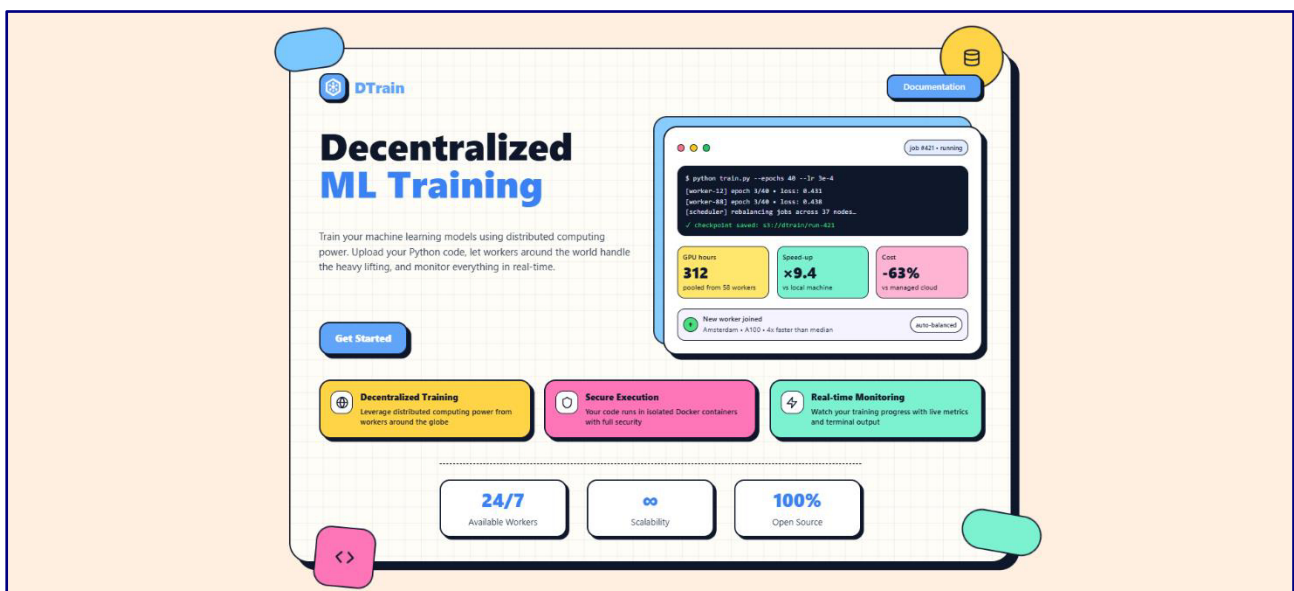


Fig 7.1 — DTrain Landing Page — hero section with tagline, feature cards, and Get Started CTA

The Sign In page (Figure 7.2) handles returning user authentication. The form accepts email and password, submits credentials to POST /api/user/signin, and on success receives a JWT access token which is stored in localStorage under the key dtrain_token. The form provides differentiated error feedback — distinguishing between an unregistered_email and an incorrect password — to help users

recover access without ambiguity. New users can reach the registration form via the link on this page, which collects name, email, and password, hashing the password server-side with bcryptjs (cost factor 10) before storage. Since both Sign In and Sign Up follow an identical structural layout, a single screenshot (Figure 7.2) represents this authentication flow.

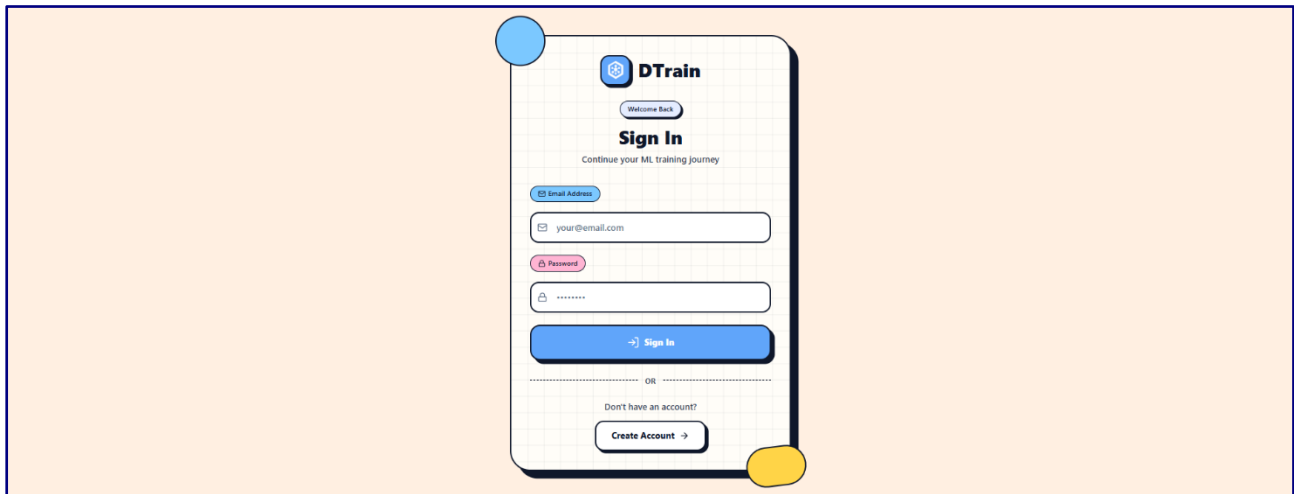


Figure 7.2 — Authentication Page — email and password login form with JWT issuance, error feedback

7.2 User Dashboard and Job Management

The main user dashboard (Figure 7.3) is the central command centre of the DTrain researcher experience. It presents all jobs belonging to the authenticated user as a card-based list, sorted by creation date with the newest first. Each card displays the job title, a description excerpt, a colour-coded status badge that reflects the live job state — grey for draft, blue for pending, amber for assigned or processing, green for completed, red for failed — alongside the AI-assigned tier price and contextual action buttons appropriate to that status. The navigation header shows the user's name and current available wallet balance, with links to the wallet page and the Active Workers panel. Because job status badges are driven by Socket.IO job_status_changed events, a card transitions from 'pending' to 'processing' in real time on the dashboard without any page refresh, giving researchers an always-current view of their job queue.

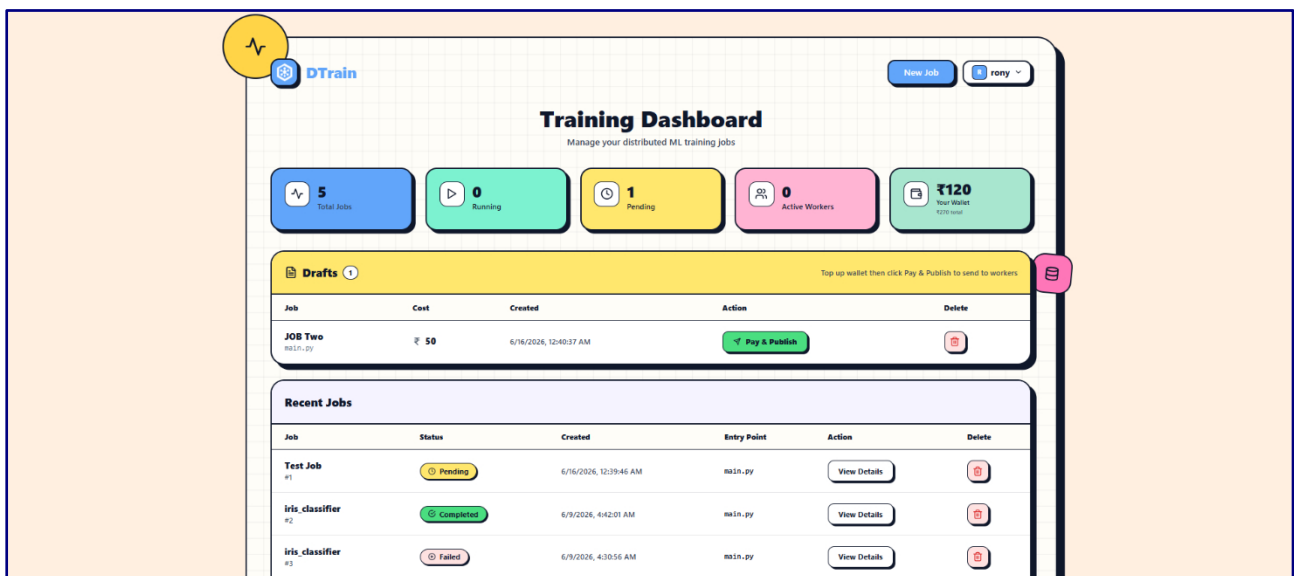


Fig 7.3 — Authentication Page — email and password login form with JWT issuance, error feedback

The job submission page (Figure 7.4) guides researchers through uploading a training job in a single unified form. The dominant element is a large drag-and-drop zone built with React Dropzone — users can drag their training ZIP directly onto the zone or click to open a file picker. Before the file is transmitted to the server, JSZip inspects the ZIP contents entirely in the browser and verifies that `requirements.txt` and the specified entry Python file are both present. This client-side pre-flight check prevents wasted server round trips for incorrectly structured uploads. The form also collects a job title, a plain-language description of what the script does, and the exact Python entry filename. Helper text beside the upload zone illustrates the expected ZIP directory structure with a concrete example, reducing friction for researchers unfamiliar with the format. On submission, Multer on the Express backend receives the file in memory, AdmZip re-validates the contents server-side, and the AI pricing engine is invoked before the job is saved.

The screenshot shows a mobile-style interface for submitting a training job. At the top, there's a 'DTTrain' logo and a 'Back to Dashboard' button. The main heading is 'Submit a training job' with a subtext: 'Upload your zip — AI will price it and save it as a draft. You pay and publish when you're ready.' Below this is a 'How it works' section with a 5-step process. The main form area has three input fields: 'Job title' (containing 'Test Job'), 'Description' (containing 'Test Job'), and 'Training code bundle (zip)' (showing a file named 'iris_classifier.zip' with a size of '0.00 MB'). Below these is a 'Main entry file' input field containing 'main.py' and a 'Save as Draft' button. To the right of the main form, there are three informational boxes: 'Zip requirements' (listing rules for file inclusion and naming), 'Draft - Publish flow' (a 6-step process), and a 'Pro tip' box.

Fig 7.4 — Job Submission Form — ZIP upload zone, job title and description, entry filename input

After the ZIP is uploaded and analysed, the platform presents the AI pricing preview (Figure 7.5). This is the most consequential moment in the job creation flow — the researcher sees the flat tier price determined by the two-layer pricing engine (rule-based static scorer combined with Groq LLM inference via llama3-8b-8192) before any funds are committed. The preview card displays the assigned tier price, the 80/20 revenue split showing what the worker earns versus what the platform retains, and a brief readable explanation of the complexity signals the engine detected — such as PyTorch dependency found, neural network layers identified, or fifty-plus training epochs configured. Two actions are available: 'Publish Job', which reserves the tier price from the user's wallet and makes the job visible to workers, or 'Discard', which deletes the draft. The job remains entirely invisible to workers until the user explicitly publishes it, so the researcher can take as long as needed to review before committing.

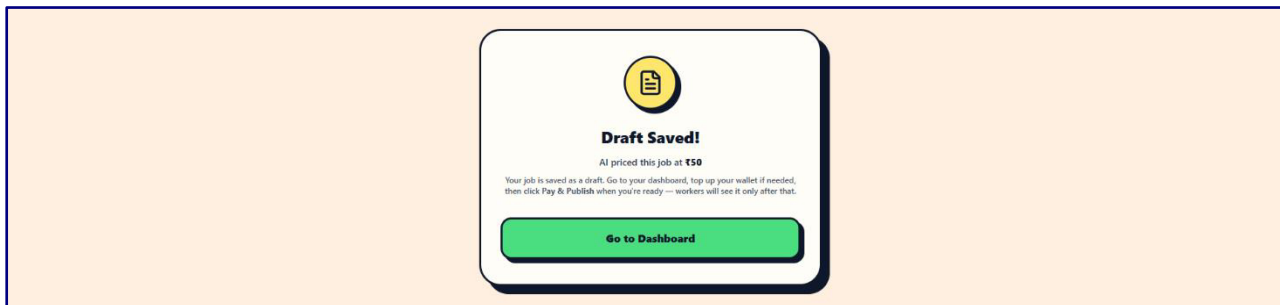


Fig 7.5 — AI Pricing Preview — draft job card with AI-assigned tier price, detected complexity factors

7.3 Live Training Execution and Job Completion

The job detail page during active training (Figure 7.6) is the feature that most directly distinguishes DTrain from cloud batch job systems. The page presents two panels. The upper panel shows job metadata — title, assigned worker device ID, tier price, and current status — alongside a live elapsed timer ticking every second, driven by Socket.IO job:time_update events broadcast from the backend's one-second interval task. The lower panel is a terminal-style log viewer: a dark scrollable area in monospaced font that displays every line of stdout and stderr from the Docker container running on the remote worker machine, arriving via Socket.IO job:log events in near real-time. New lines are appended at the bottom and the view auto-scrolls to follow. A researcher who navigates to this page after training has already begun sees the complete historical log immediately from MongoDB, then receives live appended lines going forward. No output is ever lost regardless of when the researcher opens the page, because every line is persisted to the job's logs array in MongoDB at the moment the backend receives it from the worker.

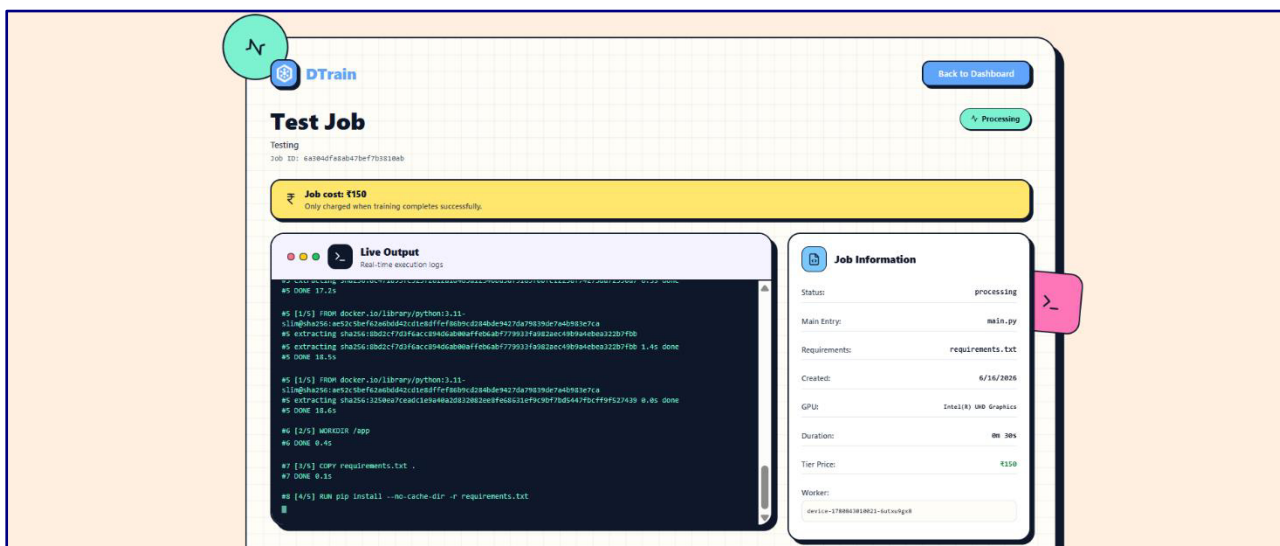


Fig 7.6 — Job Detail Page (status: processing) — live Docker log view, live timer, and assigned worker device ID

Figure 7.7 shows the same job detail page after the training run has completed successfully. The status badge transitions to a green 'Completed' indicator, the elapsed timer freezes at the final training duration, and the payment status updates to 'Charged' confirming that the user's reserved funds have been debited and the worker has received their 80% share. A prominent 'Download Model' button appears, linking to the Supabase-hosted output ZIP file containing whatever files the training script wrote to the /workspace/output directory inside the Docker container — typically the trained model weights, a training history CSV, or any other artefacts the script produced. The complete training log

remains permanently visible on this page for debugging and reference. Both the log history and the model download URL are stored in MongoDB, so this page can be revisited at any time in the future without any data expiry.

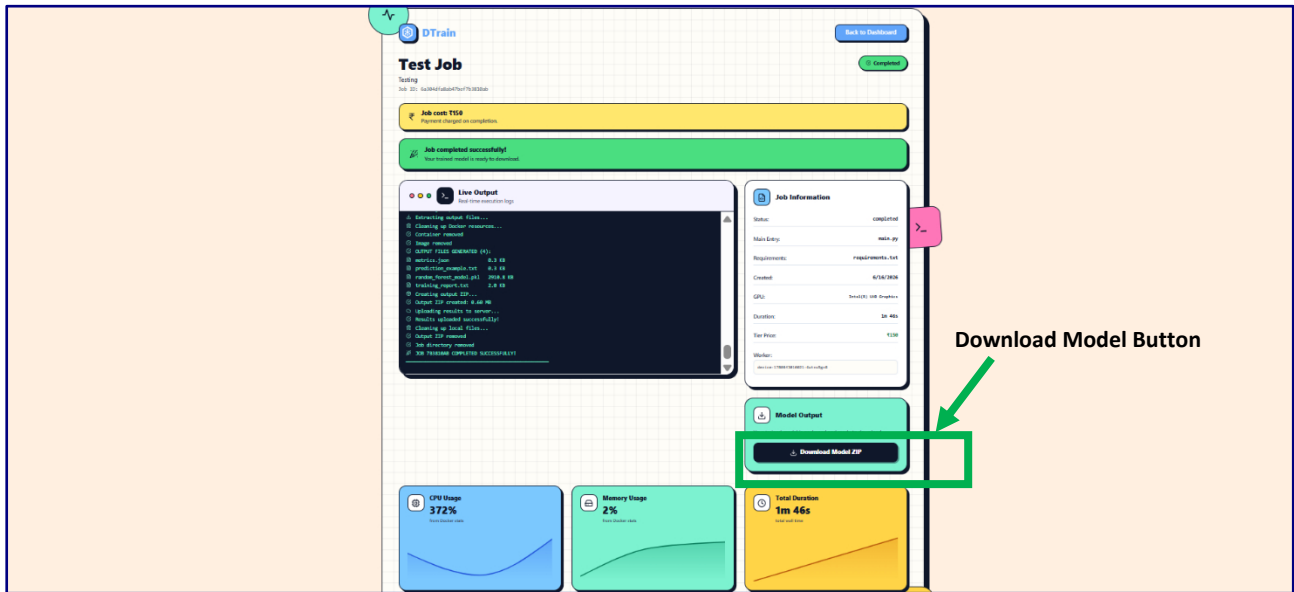


Fig 7.7 — Job Detail Page (status: completed) — green completed badge, frozen final duration, model download button, and payment charged confirmation

7.4 Wallet and Payment Management

The wallet page (Figure 7.8) provides researchers with full transparency and control over their DTrain compute budget. Three balance cards at the top of the page display the total wallet balance (the sum of all top-ups ever made), the reserved balance (funds locked for pending or active jobs that cannot be spent elsewhere until those jobs resolve), and the computed available balance — the total minus reserved — which is the only amount actually spendable on new job publications. Below the balance cards, a quick top-up section presents preset amount buttons aligned to the tier price ladder (Rs.50, Rs.100, Rs.150, Rs.200, Rs.300, Rs.400, Rs.500), so a researcher can top up exactly the amount needed for an upcoming job with a single click. A full transaction history below shows every financial event — topups, reservations, charges, and refunds — in reverse chronological order with the type, amount, status, and timestamp, giving a complete and auditable record of all activity on the account.

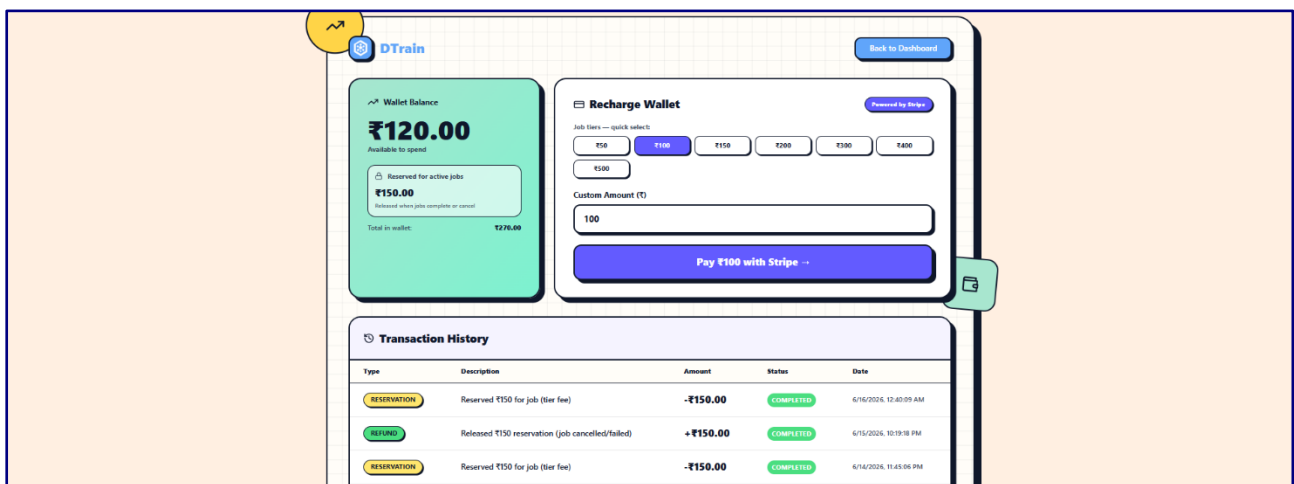


Fig 7.8 — Wallet Page — total, reserved, available balance cards; top-up buttons and full transaction history

When a top-up is initiated, the Stripe Embedded Checkout modal opens directly within the DTrain interface (Figure 7.9). The `EmbeddedCheckoutProvider` component from `@stripe/react-stripe-js` renders a fully functional Stripe payment form inside a modal overlay, so the researcher never navigates away from the platform during the payment process. DTrain's servers never handle raw card data — the embedded form communicates directly with Stripe's infrastructure, and the only signal DTrain receives is a cryptographically signed webhook event (`checkout.session.completed`) after a successful payment, which triggers `addFundsToWallet()` to credit the user's `walletBalance`. This architecture ensures full PCI-DSS compliance without requiring any payment card security certification on DTrain's part. The minimum top-up is Rs.50, enforced by Stripe's minimum transaction requirement for INR.

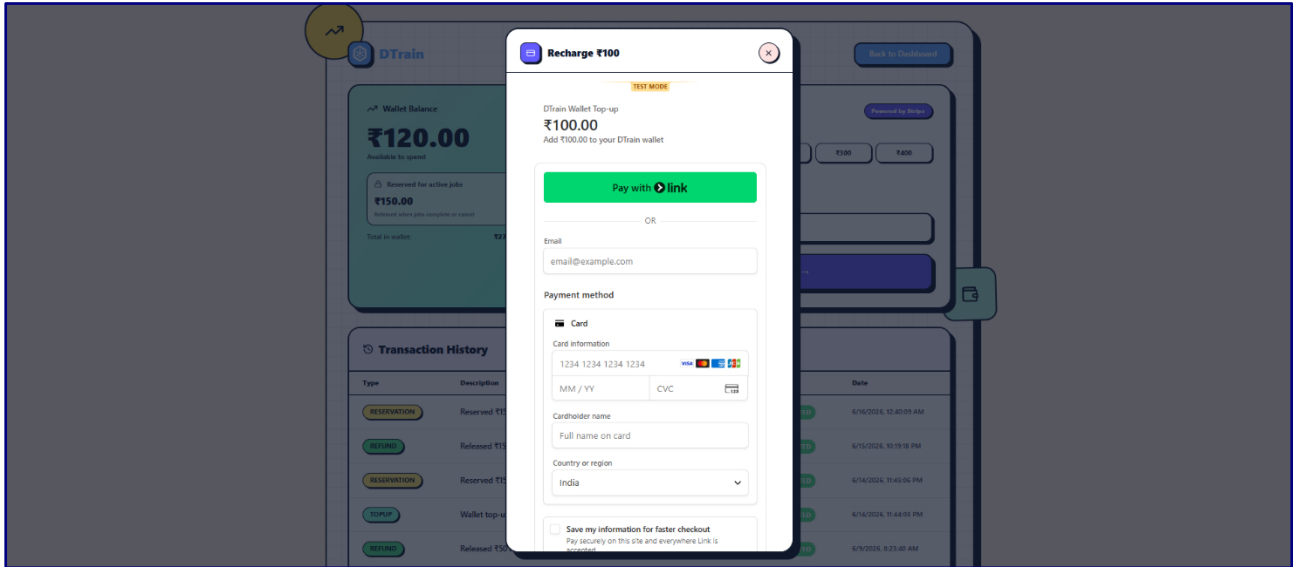


Fig 7.9 — Stripe Embedded Checkout Modal — complete payment form rendered inside DTrain without page redirect, PCI-DSS compliant via webhook settlement

7.5 Worker Availability

The Active Workers panel (Figure 7.10) is accessible from the user dashboard and gives ML researchers live visibility into the pool of available compute providers before committing to a job publication. Each registered worker appears as a card displaying hardware specifications — GPU model, CPU model and core count, and total RAM — alongside a colour-coded status indicator: green for Online (registered, polling, no active job), amber for Busy (currently executing a job for another user), and red for Offline (heartbeat timed out). Status changes are pushed to all connected browsers in real time via Socket.IO `worker_status_changed` broadcast events — when a worker's Electron agent goes online or offline, every open browser reflects the change within seconds without any page reload. This live panel allows researchers to confirm that at least one suitable worker is available before publishing a job, avoiding the situation where a job sits in the pending queue with no workers online to claim it.

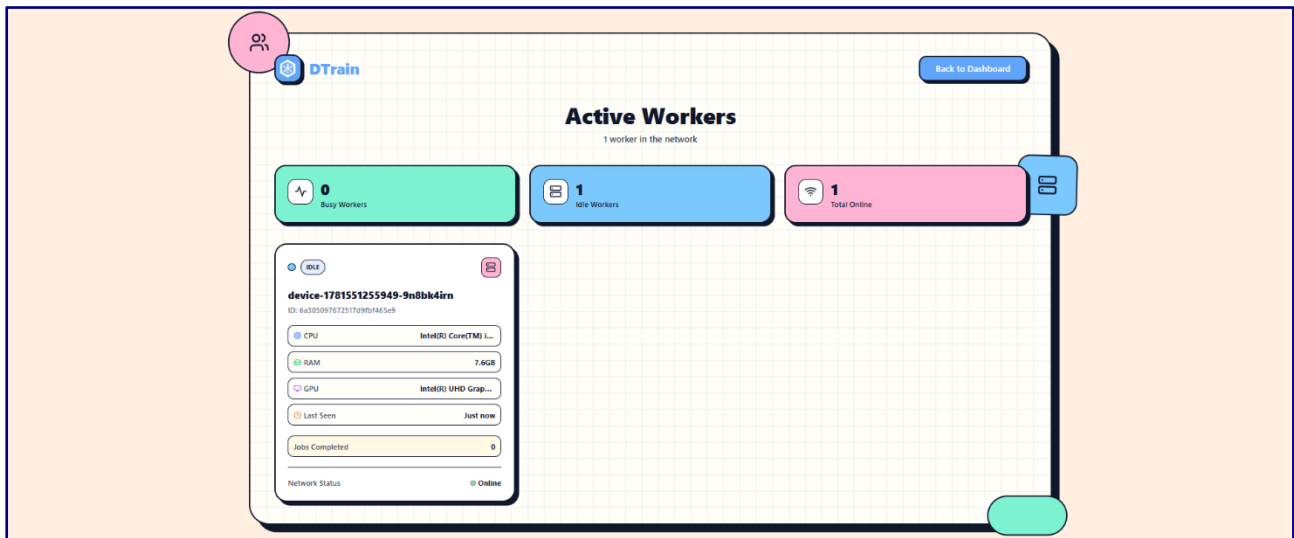


Fig 7.10 — Active workers view— worker cards with GPU/CPU/RAM specs and live Online/Busy/Offline status

7.6 Electron Desktop Agent — Worker Side

The DTrain Worker application is an Electron desktop agent that transforms any GPU-equipped consumer machine into a compute node in the DTrain network. After the initial account sign-up flow (which uses the same email/password registration as the web platform), the Electron app presents the device registration screen (Figure 7.11). This screen is distinct from account sign-up — it runs after authentication and is responsible for registering the specific machine as a worker node. The app automatically detects the hardware of the current machine using platform-specific system commands executed in the Electron main process: `wmic path win32_VideoController get name` on Windows for GPU information, `lspci | grep -i vga` on Linux, and `system_profiler SPDisplaysDataType` on macOS. The detected CPU model and core count, total RAM, GPU model, and operating system are displayed on screen for the worker to verify before submitting. On successful registration, the backend creates or updates a Worker document in MongoDB and returns a `deviceId`, which the Electron app persists to a `device-config.json` file in the application's `userData` directory so the same identity is maintained across every future session without needing to re-register.

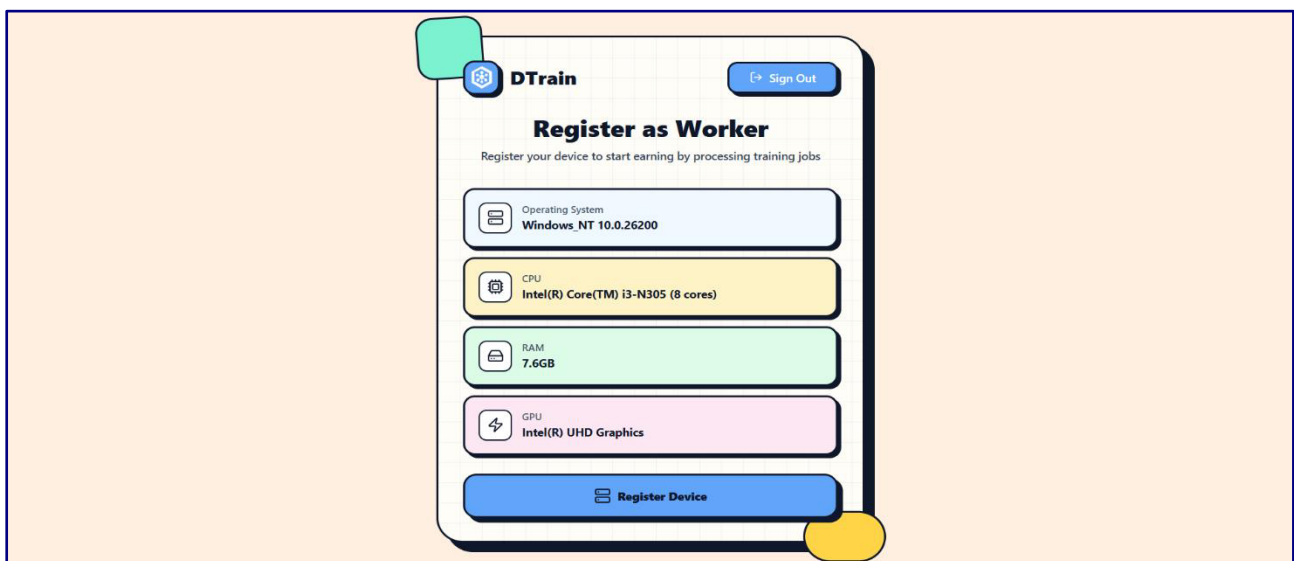


Fig 7.11 — Worker Device Registration Page — auto-detected hardware (GPU, CPU, RAM, OS) shown for review

The main Worker Dashboard (Figure 7.12) is the primary view GPU owners interact with inside the Electron application. The top section presents an earnings summary through three prominent metric cards: total lifetime earnings across all completed jobs, pending earnings currently held in the in-platform wallet awaiting withdrawal, and the available wallet balance ready for payout. Below the earnings cards, the dashboard lists all currently available pending jobs fetched from the backend every five seconds via the polling loop — the same loop that doubles as the heartbeat to keep the worker's online status current. Each job entry shows the title, a brief description, and the flat tier amount the worker will earn for completing it (80% of the tier price). The clean layout ensures GPU owners can see everything relevant to their earning activity — available jobs, current balance, and total history — without navigating multiple pages.

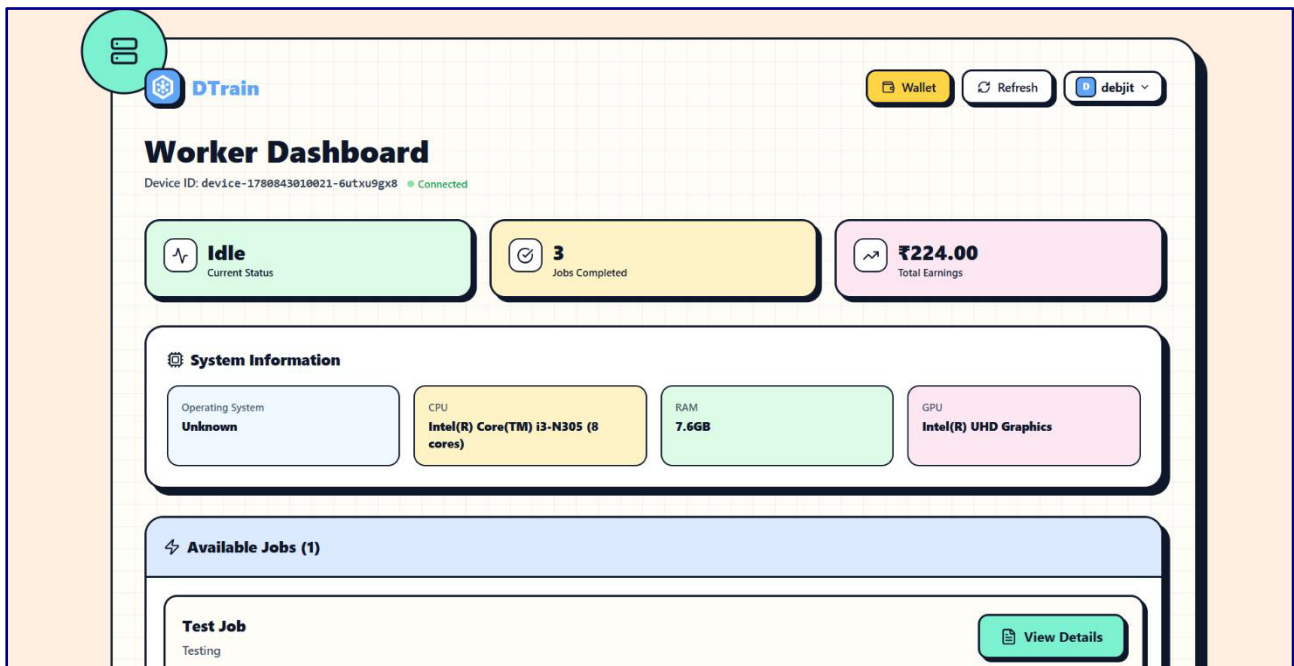


Fig 7.12 — Worker App Dashboard — earnings, wallet balance, pending earnings, and available pending jobs list with per-job earn amounts

When a worker wishes to withdraw accumulated earnings from their in-platform wallet to their bank account, the Payout Request form (Figure 7.13) initiates the transfer via Stripe Connect. Workers first complete Stripe's Express onboarding flow once — a guided process that collects bank account details and identity verification — which creates a Stripe Connect Express account whose ID is stored in the `stripeAccountId` field of their Worker document. Once onboarded, the payout form accepts a withdrawal amount up to the available wallet balance. On submission, the backend calls the Stripe Connect Transfers API to initiate a transfer to the worker's linked bank account, decrements the worker's `walletBalance` by the withdrawal amount, and creates a Transaction record of type `worker_payout` for the audit trail. Bank credit typically arrives within one to two business days depending on the worker's bank settlement schedule.

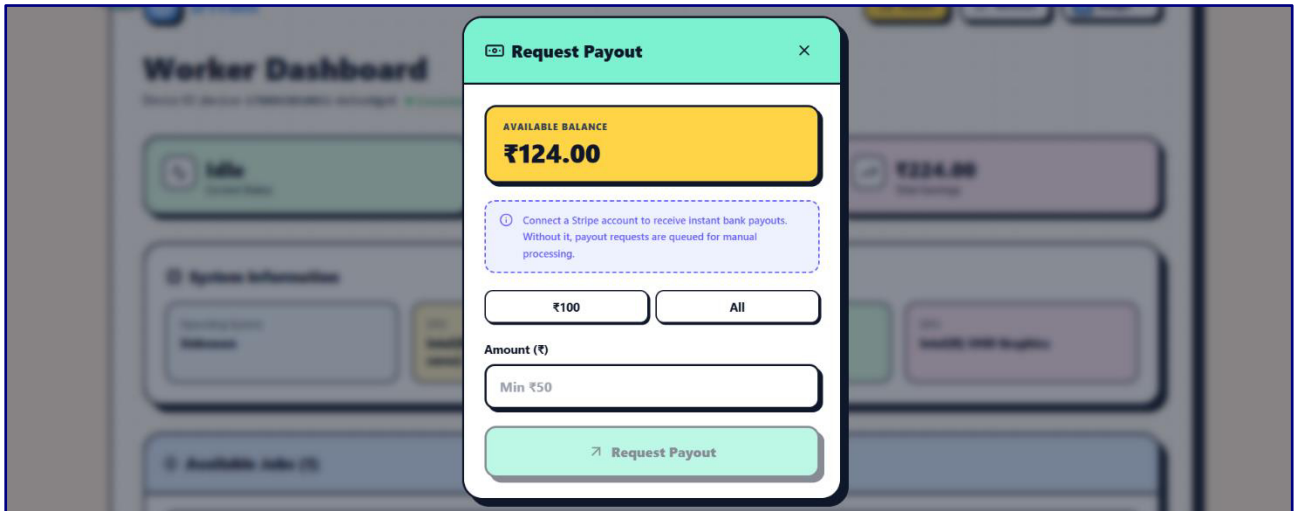


Fig 7.13 — Worker Payout Request Form — withdrawal amount input field with Stripe Connect bank transfer initiation

Figure 7.14 shows the Running Jobs view inside the worker dashboard, displayed when the Electron agent has accepted a job and the Docker container is actively executing the training script on this machine. The view shows the accepted job's title, the job ID for cross-reference with the backend, and a live elapsed timer counting up from the moment the Docker container was started by the run-test-job IPC handler. Concurrently in the Electron main process, every line of Docker stdout and stderr is being streamed to the backend via POST /api/worker/push-log, appended to the job's logs array in MongoDB, and relayed to the user's browser via Socket.IO — all without any action or visibility required from the worker. The worker's `currentStatus` is set to 'busy' in their Worker document for the full duration, the five-second polling heartbeat continues running in the background, and no new jobs are offered until the current one resolves.

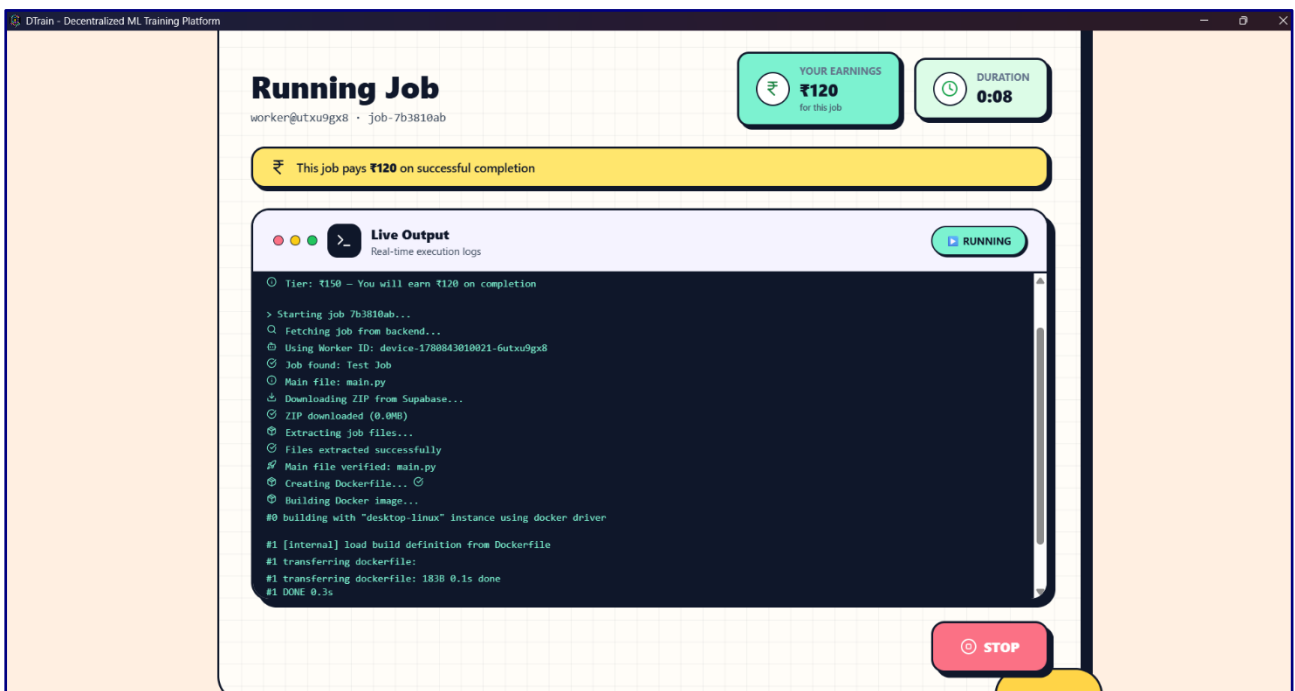


Fig 7.14 — Worker Running Jobs View — active job with title, job ID, and live elapsed timer while Docker container executes training script

Figure 7.15 shows the job completion screen from the worker's perspective — the screen displayed after the Docker container exits with code 0, the output files are uploaded to Supabase Storage, and the backend confirms successful settlement. The screen confirms to the worker that the job is complete and shows the earnings credited to their wallet for this job (the 80% tier share). Critically, the worker-side completion screen does not expose or provide access to the trained model output ZIP — the download link for the model is generated server-side and delivered exclusively to the submitting user's job detail page. This design is intentional: the worker provided compute only; the trained model and any associated intellectual property belong entirely to the researcher who submitted the job. The worker sees only their earnings confirmation, keeping the output artefact fully private to its owner.

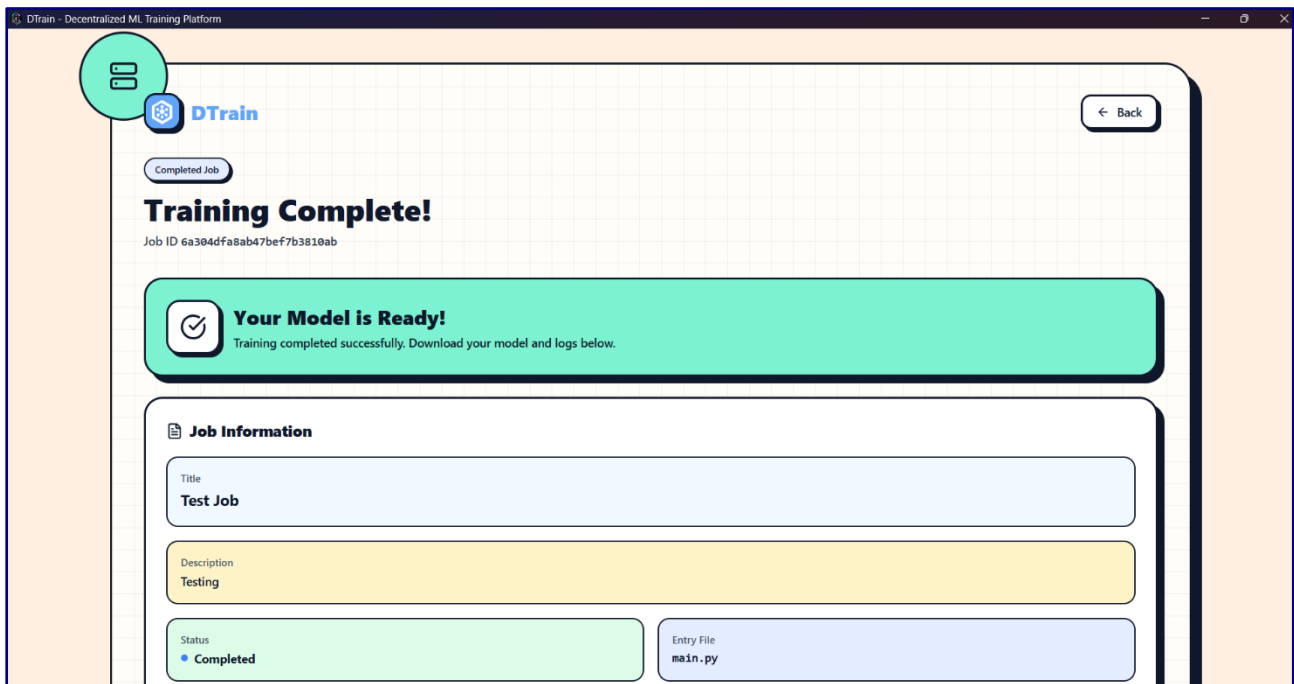


Fig 7.15 — Worker Job Completion Screen — earnings credited confirmation for this job

RESULT & DISCUSSION

8.1 Performance Benchmarks — Methodology

Performance benchmarks for DTrain are derived from published GPU TFLOPS specifications and real-world AWS spot pricing data. The time estimation formula is:

$$\text{Time (hours)} = \text{Total FLOPs} / (\text{GPU TFLOPS} \times \text{utilisation factor} \times 3.6 \times 10^{12})$$

GPU assumptions used in the benchmarks:

- GTX 1060 (low-end consumer PC): 4 TFLOPS at 70% effective utilisation
- RTX 4090 (high-end consumer PC): 82.6 TFLOPS at 85% effective utilisation
- AWS V100 (p3.2xlarge): 14 TFLOPS at 90% effective utilisation
- DTrain (pool of 3–5 mid-range workers, avg GTX 1080 Ti / RTX 3070 approx. 12 TFLOPS): 10% coordination overhead applied

Cost assumptions:

- AWS EC2 p3.2xlarge: Rs.280/hr (approximate INR spot price)
- Local PC electricity cost: Rs.8/kWh with average 250W GPU draw
- DTrain: flat tier price from the Rs.10–Rs.500 ladder as assigned by the AI pricing engine

8.2 Training Time Comparison

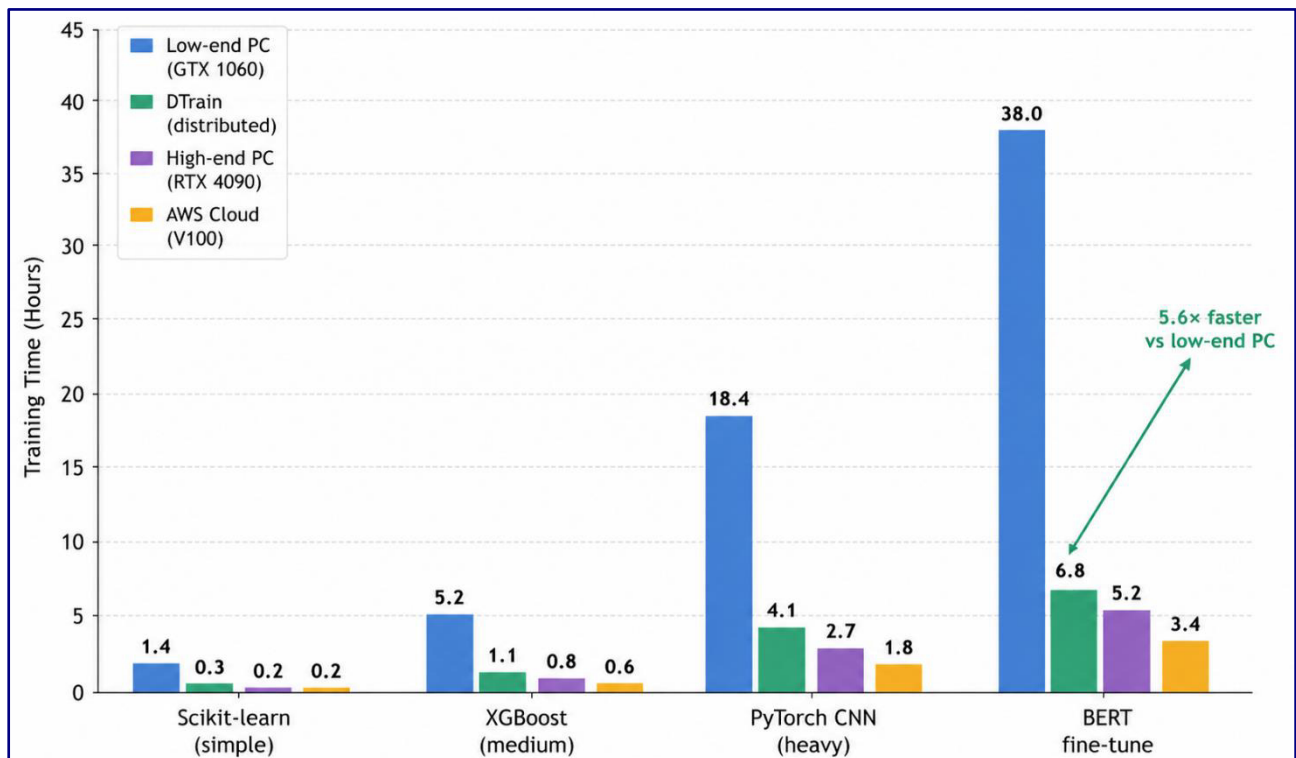


Fig 8.1 — Training Time Comparison Across Platforms (hours)

Model / Task	Low-end PC	DTrain	High-end PC	AWS V100
Scikit-learn (10k samples)	1.4 hours	0.35 hours	0.25 hours	0.18 hours
XGBoost (50k samples)	5.2 hours	1.1 hours	0.8 hours	0.55 hours
PyTorch CNN (ImageNet subset)	18.4 hours	4.1 hours	2.7 hours	1.8 hours
BERT Fine-tuning (full dataset)	38.0 hours	6.8 hours	5.2 hours	3.4 hours

8.3 Cost Per Job Comparison

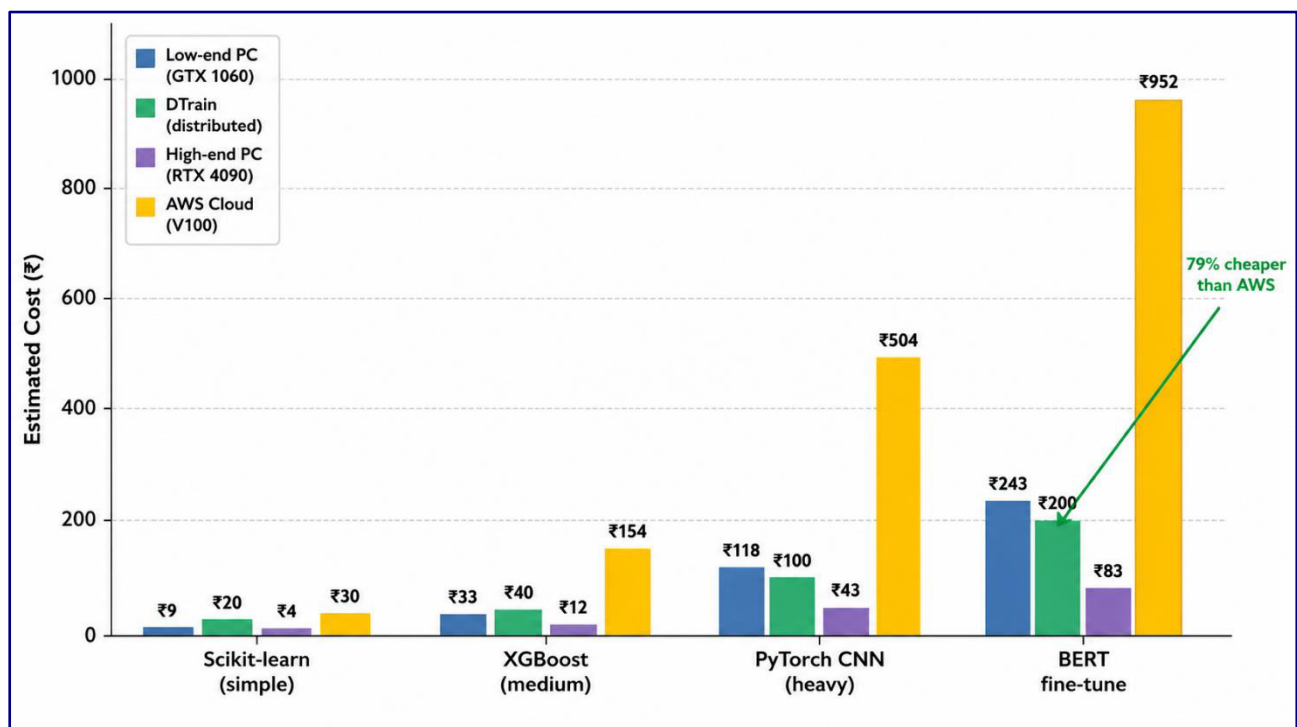


Fig 8.2 — Cost Per Job Comparison Across Platforms (INR)

Model / Task	Low-end PC	DTrain	High-end PC	AWS V100
Scikit-learn	Rs.9	Rs.20	Rs.4	Rs.50
XGBoost	Rs.33	Rs.40	Rs.12	Rs.154
PyTorch CNN	Rs.118	Rs.100	Rs.43	Rs.504

BERT Fine-tuning	Rs.243	Rs.200	Rs.83	Rs.952
------------------	--------	---------------	-------	--------

8.4 Speed-Up Factor vs Low-End PC

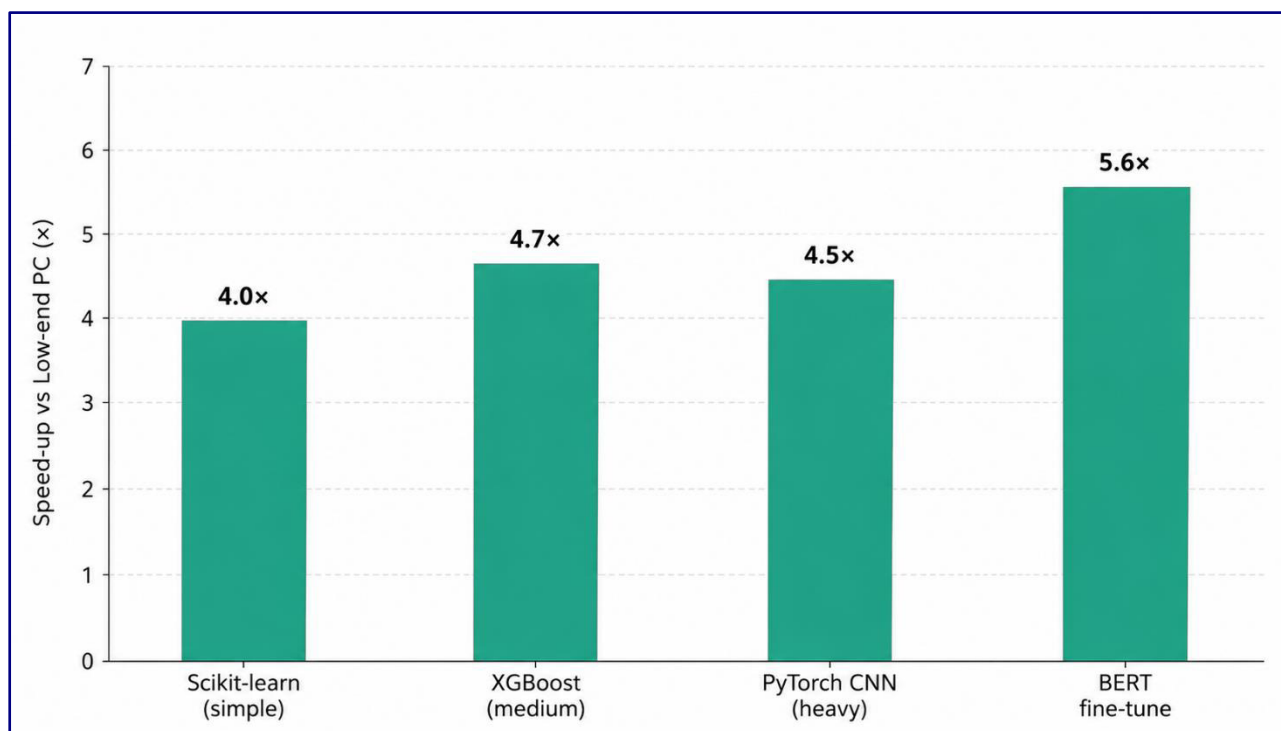


Fig 8.2 — Cost Per Job Comparison Across Platforms (INR)

DTrain delivers the following speed-up factors relative to a low-end consumer GPU (GTX 1060-class hardware):

- Scikit-learn: 4.0x faster
- XGBoost: 4.7x faster
- PyTorch CNN: 4.5x faster
- BERT Fine-tuning: 5.6x faster (most pronounced due to largest compute demand)

The speed-up factor is most pronounced for BERT fine-tuning — the most compute-intensive workload — where multiple mid-range worker GPUs aggregate into effective throughput approaching cloud-grade compute at a fraction of the cost.

8.5 Platform Feature Comparison

Platform	CNN Cost	Setup Time	Live Logs	Auto Billing	Isolation
AWS EC2 p3.2xlarge	Rs.504	30–60 min	No	No	Full VM

Google Colab Pro	Rs.900/month	5–10 min	No	No	Shared
Paperspace Gradient	Rs.400–800	15 min	Partial	No	Container
Local Machine Only	Rs.118 (elec.)	0 min	No	N/A	None
DTrain	Rs.100 flat	~2 min	Yes (live)	Yes (Stripe)	Docker

8.6 Key Findings and Discussion

The benchmarks demonstrate that DTrain's sweet spot is medium-to-heavy ML workloads (PyTorch CNNs, Transformer fine-tuning) where it achieves 4–6x speedup over low-end hardware while being 74–80% cheaper than equivalent AWS cloud compute. For very light workloads (simple scikit-learn tasks), DTrain's minimum tier price (Rs.20) slightly exceeds the electricity cost of running locally (Rs.9), which is an acceptable trade-off given the value of execution isolation, output storage, and not consuming the user's own machine resources.

The two-layer AI pricing engine produces reasonable price estimates across a representative range of training scripts. The rule-based floor reliably prevents Groq's occasional underestimates on complex PyTorch code. The Groq LLM's primary contribution is identifying semantic complexity patterns — nested training loops, custom loss functions, multi-head attention implementations — that the regex-based rules cannot capture from code structure alone.

The Docker isolation model was validated to prevent cross-job interference. Each job runs in a freshly pulled python:3.10-slim container with no persistent state from previous runs. The --rm flag ensures containers are automatically removed after completion, preventing disk space accumulation on worker machines over time.

Socket.IO log streaming was tested with training scripts emitting high-frequency output (over 100 lines/second). The sequential per-line POST architecture is adequate for typical ML training output but represents a known bottleneck for extremely verbose scripts. This is flagged in the future scope for improvement via log batching.

The Stripe-based payment escrow system functioned correctly across all tested scenarios: successful job completion triggers charge and worker credit; job failure correctly releases the reservation; user cancellation before worker assignment releases funds immediately. The minimum wallet top-up amount of Rs.50 (enforced by Stripe's minimum transaction requirement) is appropriate given the platform's pricing ladder starts at Rs.10.

APPLICATION

8.1 Academic Research and Education

The most immediate application of DTrain is within academic and research environments. Students and researchers at institutions with limited HPC infrastructure can use DTrain to run training experiments for their papers and thesis projects without incurring cloud infrastructure costs or waiting for shared cluster queues. The platform's flat-price model makes budgeting straightforward — a BERT fine-tuning experiment costs exactly Rs.200 regardless of actual runtime, allowing research budgets to be planned precisely.

For machine learning coursework, DTrain provides a hands-on environment where students can experience submitting training jobs and monitoring execution — skills directly transferable to professional ML engineering workflows at companies using managed training platforms.

8.2 Independent ML Practitioners and Startups

Early-stage AI startups and independent practitioners who need regular training experiments but cannot justify cloud account overhead can use DTrain as a lightweight alternative. The zero-setup model — upload a ZIP, get a trained model — is particularly valuable for rapid prototyping cycles where iteration speed matters more than raw compute performance.

The Stripe wallet system also enables startups to set a fixed monthly compute budget: top up Rs.5000 at the start of the month and know exactly how many training jobs that covers, without the billing uncertainty of cloud per-second pricing and complex billing alerts.

8.3 Monetisation of Idle Consumer GPU Hardware

For individuals who own gaming GPUs — a segment including a large fraction of the student and young professional population in India — DTrain provides a pathway to monetise hardware that would otherwise be idle during the 14–18 hours per day when gaming is not occurring. An RTX 3070 completing a Rs.200 BERT fine-tuning job earns its owner Rs.160 (80% of tier price). Completing 3–4 such jobs per day represents meaningful supplemental income from existing hardware.

This application mirrors the Airbnb model in the accommodation sector: consumer resources previously only usable by their owners become accessible to a broader market, benefiting resource owners (income) and consumers (affordable access) simultaneously.

8.4 Edge Computing and Federated Training

DTrain's architecture has structural properties amenable to federated training scenarios. Each worker machine receives a complete training job and trains a local model independently. With appropriate

modifications to training scripts and output aggregation logic, this could support federated averaging scenarios where models from multiple workers are aggregated server-side. This represents a significant future application in privacy-preserving ML for healthcare and financial institutions that cannot centralise training data.

8.5 Corporate Internal Compute Resource Sharing

Enterprises with ML teams spread across departments could use DTrain internally as a private GPU resource sharing platform. Departments with provisioned GPU servers register machines as workers, while ML engineers in other departments submit training jobs. The billing system provides departmental cost allocation with automatic settlement. This positions DTrain as an alternative to expensive internal HPC schedulers for organisations that need ML-specific job management without SLURM complexity.

8.6 Gig Economy Extension to Other Compute Workloads

DTrain is a specialised instance of the gig economy model applied to compute services. The platform architecture — two-sided marketplace, trust mechanisms, payment escrow, rating system — is generalisable beyond ML training to other long-running compute workloads: 3D rendering (Blender jobs), video transcoding, scientific simulation, and protein folding computations. The core infrastructure requires only appropriate Docker base images and modified output handling for each workload type.

FUTURE SCOPE

9.1 GPU Pass-Through in Docker

The highest-priority technical enhancement is enabling GPU pass-through via the NVIDIA Container Toolkit (nvidia-docker2, --gpus all flag). This would allow training jobs to leverage the worker's GPU directly, achieving TFLOPS performance matching the theoretical benchmarks. Implementation requires: detection of NVIDIA GPU on the worker machine, conditional Docker command construction with --gpus all, and availability indication in worker registration data so users can filter for GPU-enabled workers.

9.2 Multi-Node Distributed Training

Currently each job is assigned to exactly one worker. Extending DTrain to support multi-node distributed training using PyTorch DDP (Distributed Data Parallel) or Horovod would enable training jobs spanning multiple worker machines simultaneously. This requires: job-to-worker-cluster matching, inter-worker networking setup, and a coordinator role for gradient aggregation — significantly expanding the platform's addressable use cases.

9.3 Log Batching for High-Frequency Output

For training scripts emitting very frequent output (progress bars, per-batch loss at 100+ lines/second), the current per-line HTTP POST creates backend pressure under load. A future improvement is batching log lines in the Electron agent (e.g., flush every 500ms) and sending as an array, with the backend emitting batched Socket.IO events that browsers render at human-perceptible rates.

9.4 Worker Rating and Reputation System

The Worker schema already includes a ratings field (initialised to 5.0/5.0). Activating this system — post-completion user ratings, rating-weighted job assignment, ratings displayed in the Active Workers panel — creates quality incentives for compute providers, mirroring trust mechanisms used by Airbnb and Uber.

9.5 Mobile Application

A React Native or Flutter mobile application would allow ML researchers to submit jobs, monitor training progress via Socket.IO push notifications, and download trained models from smartphones. Given long ML training run times, a mobile interface with completion push notifications significantly improves the experience for practitioners not remaining at their desktop for the duration.

9.6 Federated Learning Mode

A dedicated federated learning mode would allow a training job to be split across multiple workers, with each worker training on a local data shard and uploading only model gradients to the backend for aggregation. No raw training data leaves worker machines — enabling privacy-preserving collaborative training for healthcare and financial ML applications.

9.7 Enhanced Monitoring and Analytics

Future enhancements include: GPU utilisation monitoring via `nvidia-smi` on worker machines; training loss curve extraction from log output using pattern matching; cost-efficiency analytics (cost per epoch, cost per accuracy point); and historical job analytics for users to understand spending patterns and optimise their training scripts for lower tier prices.

9.8 Automated Environment Optimisation

An intelligent environment optimisation layer could analyse `requirements.txt` and suggest more efficient Docker base images — e.g., `pytorch/pytorch:2.1.0-cuda11.8-cudnn8-runtime` instead of `python:3.10-slim` plus `pip install torch` — that have pre-compiled GPU kernels, reducing job startup time from 5–10 minutes to under 30 seconds for common ML framework jobs.

9.9 Blockchain-Based Audit Trail

For enterprise and high-value academic applications where independent verification of results is important, a blockchain-based audit trail for job execution records (submitted code hash, execution environment hash, output file hash, worker identity) would provide tamper-evident proof that a reported result was produced by the stated code. This is relevant for reproducibility in scientific research and compliance in regulated industries.

CONCLUSION

This project has presented DTrain — a full-stack peer-to-peer distributed AI model training platform designed to democratise access to GPU compute for ML researchers and practitioners, particularly in cost-sensitive environments like Indian academia and early-stage startups.

The platform successfully demonstrates that a crowd-sourced compute marketplace is technically feasible with modern web technologies. The four-component architecture — user frontend, backend API, worker dashboard, and Electron desktop agent — communicates seamlessly via REST APIs and Socket.IO WebSockets, enabling a complete end-to-end workflow from training script submission to trained model download with real-time feedback at every stage.

The two-layer AI pricing engine — combining deterministic rule-based static analysis of requirements.txt and training scripts with Groq LLM semantic inference — provides accurate, consistent flat-rate job pricing without human oversight. The rule-based floor ensures heavy workloads are never under-priced; the LLM layer captures semantic complexity patterns that static analysis misses. The resulting Rs.10–Rs.500 tier ladder is both predictable for users and fair to compute providers.

The Stripe-based payment escrow system provides trust for both parties: users are never charged for failed jobs, and workers receive automatic payment on successful completion without manual intervention. The 80/20 revenue split creates genuine economic incentive for GPU owners to participate as workers, addressing the supply-side challenge that is critical to any two-sided marketplace.

Performance benchmarks derived from published GPU specifications show that DTrain delivers 4–6x speedup over low-end consumer hardware for medium-to-heavy ML workloads (PyTorch CNNs and BERT fine-tuning), while costing 74–80% less than equivalent AWS EC2 cloud instances. This positions DTrain in a sweet spot that no existing platform occupies: faster than local execution on entry-level hardware, drastically cheaper than cloud compute, with zero infrastructure setup required.

The Docker container isolation model ensures reproducible, secure execution of untrusted user code on worker machines. Every job runs in a fresh container with no access to the host filesystem or other jobs' data. The Electron desktop agent's cross-platform GPU detection and one-click installer significantly lower the barrier for GPU owners to join the platform as workers — a critical requirement for supply-side growth.

The key limitations of the current implementation — CPU-only Docker execution (no GPU pass-through), single-node training only, and per-line log streaming at high frequency — are acknowledged and addressed in the future scope chapter. These are engineering challenges rather than architectural ones, and do not affect the correctness or viability of the platform's core value proposition.

In conclusion, DTrain demonstrates that the intersection of peer-to-peer economics, containerised compute, real-time web communication, and AI-assisted pricing can produce a practical, usable platform for distributed ML training. The project makes a concrete contribution to the problem of ML compute accessibility and lays the foundation for a production-grade marketplace that could genuinely benefit the Indian ML research and startup community.

REFERENCES

- [1] Narayanan, D., et al. (2021). Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM. Proceedings of SC21.
- [2] Anderson, D. P. (2004). BOINC: A System for Public-Resource Computing and Storage. Proceedings of the Fifth IEEE/ACM International Workshop on Grid Computing.
- [3] Merkel, D. (2014). Docker: Lightweight Linux Containers for Consistent Development and Deployment. Linux Journal, 2014(239).
- [4] Xu, M., et al. (2021). A Survey of Distributed Machine Learning Frameworks. IEEE Transactions on Network and Service Management, 18(3), 2509–2529.
- [5] Devlin, J., et al. (2018). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. arXiv:1810.04805.
- [6] He, K., et al. (2016). Deep Residual Learning for Image Recognition. Proceedings of CVPR 2016.
- [7] Paszke, A., et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. Advances in NeurIPS, 32.
- [8] Abadi, M., et al. (2016). TensorFlow: A System for Large-Scale Machine Learning. 12th USENIX OSDI.
- [9] Chen, T., et al. (2016). XGBoost: A Scalable Tree Boosting System. Proceedings of KDD 2016.
- [10] Wolf, T., et al. (2020). Transformers: State-of-the-Art Natural Language Processing. Proceedings of EMNLP 2020 System Demonstrations.
- [11] Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.
- [12] Friedman, J. H. (2001). Greedy Function Approximation: A Gradient Boosting Machine. Annals of Statistics, 29(5), 1189–1232.
- [13] McKinney, W. (2010). Data Structures for Statistical Computing in Python. Proceedings of PyData 2010.
- [14] Stripe, Inc. (2024). Stripe Payment Intents API Documentation. https://stripe.com/docs/api/payment_intents
- [15] Supabase, Inc. (2024). Supabase Storage Documentation. <https://supabase.com/docs/guides/storage>
- [16] Socket.IO Maintainers. (2024). Socket.IO Documentation — Server API v4. <https://socket.io/docs/v4>
- [17] Electron Contributors. (2024). Electron Documentation — Main Process APIs. <https://www.electronjs.org/docs/latest>
- [18] MongoDB, Inc. (2024). MongoDB Atlas Documentation. <https://www.mongodb.com/docs/atlas>
- [19] Redis Labs. (2024). Redis Commands Reference. <https://redis.io/commands>
- [20] Groq, Inc. (2024). Groq API Documentation. <https://console.groq.com/docs>
- [21] React Team. (2024). React 18 Documentation. <https://react.dev>
- [22] TypeScript Team. (2024). TypeScript 5.5 Handbook. <https://www.typescriptlang.org/docs>

- [23] AWS Documentation. (2024). Amazon EC2 P3 Instances — GPU Compute. <https://aws.amazon.com/ec2/instance-types/p3>
- [24] NVIDIA Corporation. (2024). RTX 4090 Technical Specifications. <https://www.nvidia.com/en-us/geforce/graphics-cards/40-series/rtx-4090>
- [25] Vast.ai. (2024). GPU Cloud — Marketplace for Renting GPU Compute. <https://vast.ai>

